

第五章 死锁与饥饿

- 死锁
 - 竞争资源引起
 - 无限等待，可检测
- 饥饿
 - 资源分配策略不公平引起
 - 不一定处于等待状态，可能忙等

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁 的静态分析

5.14 同种组合资源死锁 的必要条件

5.1 死锁的概念





例子: $r1$ 和 $r2$ 为可再用资源;

P1: apply for $r1$;

...

① → apply for $r2$;

...

return $r1$;

...

return $r2$;

P2: apply for $r2$;

...

② → apply for $r1$;

...

return $r1$;

...

return $r2$;

死锁定义

- 一组进程中的每一个进程，均无限期地等待此组进程中其它进程所占有的，因而永远无法得到的资源，这种现象称为进程死锁
- 当死锁发生后，参与死锁的进程将一直等待下去，除非有来自参与死锁进程之外的某种干预
- 定义死锁时刻
 - 无限等待发生时
 - 等待发生前（已注定死锁）

由定义得到的结论

- 关于死锁的一些结论
 - 参与死锁的进程至少有二个
 - 每个参与死锁的进程均等待资源
 - 参与死锁的进程中至少有两个进程占有资源
 - 死锁进程是系统中当前进程集合的一个子集

死锁产生原因

- 时间上
 - 由于进程运行过程中推进顺序不当, 即调度时机不合适, 不该切换的进程进行了切换, 可能造成死锁
- 空间上
 - 对独占资源分配不当, 互斥资源部分分配又不可剥夺, 极易造成死锁

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

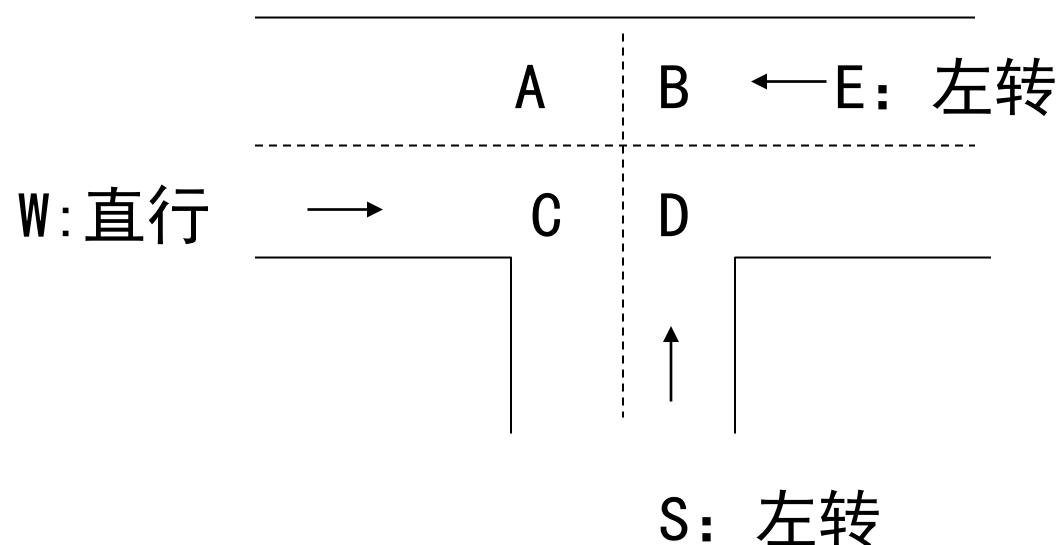
5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.2 死锁类型

1. 竞争资源引起的死锁

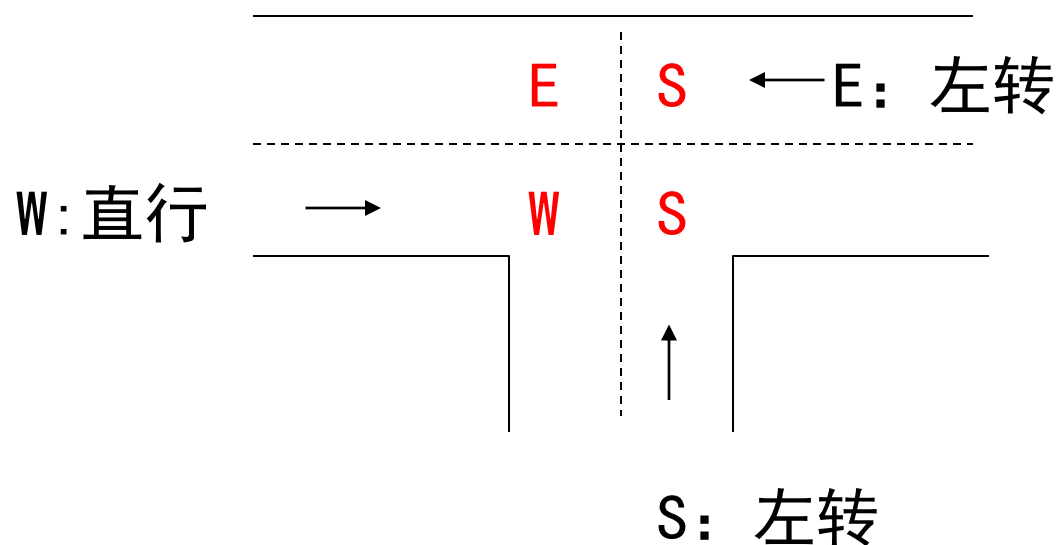
(1) 不同种资源



5.2 死锁类型 (Cont.)

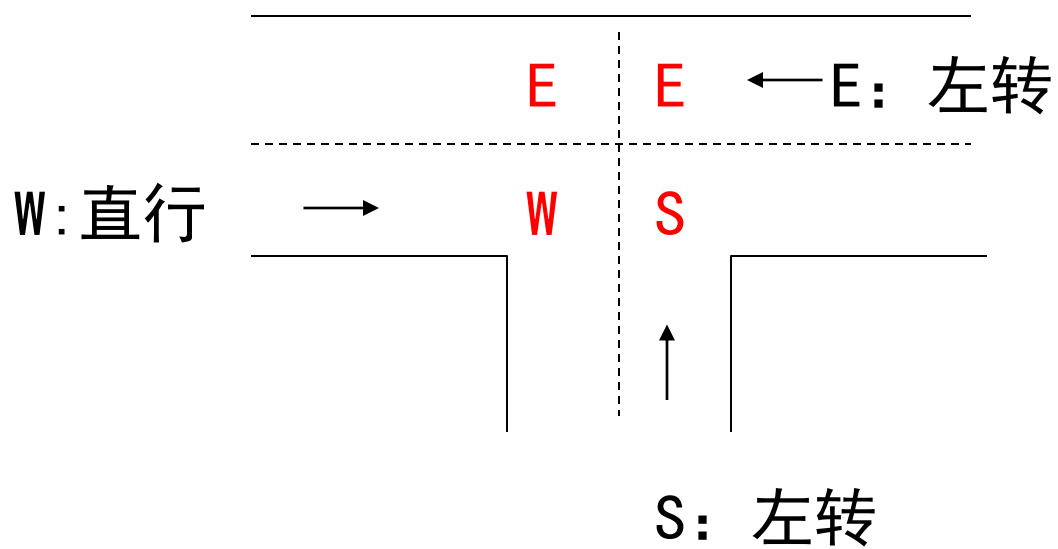
1. 竞争资源引起的死锁

(1) 不同种资源



5.2 死锁类型 (Cont.)

- 1. 竞争资源引起的死锁
 - (1) 不同种资源



5.2 死锁类型 (Cont.)

(2) 同种资源

4台打印机，申请： a ，释放 $\bar{a}$

P1: $a \ a \ a \ a \ \bar{a} \ \bar{a} \ \bar{a} \ \bar{a}$

P2: $a \ a \ \bar{a} \ \bar{a}$

5.2 死锁类型 (Cont.)

2. 进程通讯引起的死锁

P1: receive (P2, M1);

P2: receive (P3, M2);

P3: receive (P1, M3);

3. 其它原因引起的死锁

After you / after you

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.3 死锁的条件

- Coffman条件(必要条件)
 - 资源独占(mutual exclusion)
 - 不可剥夺(non preemption)
 - 保持申请(hold-while-applying)
 - 循环等待(circular wait)
- 当每类资源只有一个实例时, Coffman条件为充要条件
- 破坏上述任意一个条件可以消除死锁

死锁的处理

- 死锁预防 (deadlock prevention) —静态
- 死锁避免 (deadlock avoidance) —动态
- 死锁检测 (deadlock detection)
- 死锁恢复 (deadlock recovery)

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.4 资源分配图

定义：一个系统资源分配图是一个二元组 $G=(V, E)$ ，其中 V 是节点集， E 是边集。 $V=P\cup R$ ， $P=\{p_1, p_2, \dots, p_n\}$ ， $R=\{r_1, r_2, \dots, r_m\}$ ，

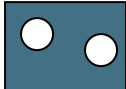
$$E=\{(p_i, r_j)\} \cup \{(r_j, p_i)\}, \quad p_i \in P, \quad r_j \in R.$$

申请边 (p_i, r_j) ： p_i 申请 r_j ；

分配边 (r_j, p_i) ： r_j 分配 p_i ；

图示：

进程： 

资源： 

申请边：由进程到资源类；

分配边：由资源实例到进程。

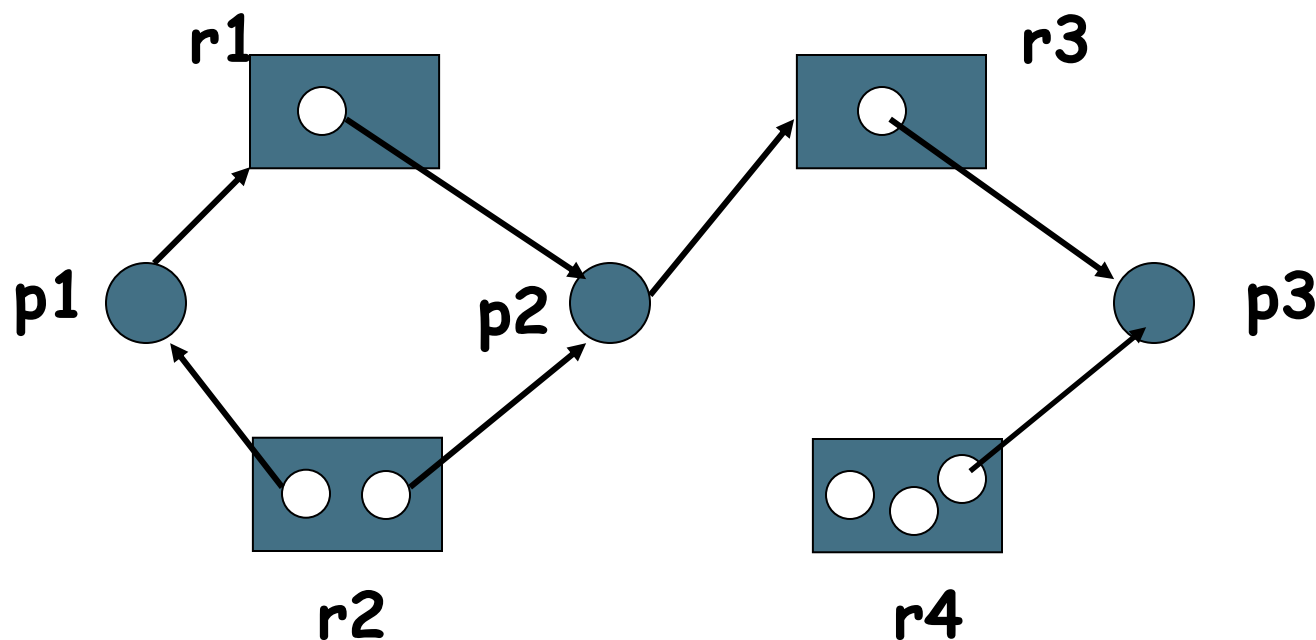
5.4 资源分配图(Cont.)

- 申请： p_i 申请 r_j 中的一个资源实例，由 p_i 向 r_j 画一条申请边，如可满足，改为分配边
- 释放：去掉分配边
- 如果资源分配图中没有环路，则系统中没有死锁；如果存在环路，则系统中可能存在死锁。

例子（无环路，无死锁）

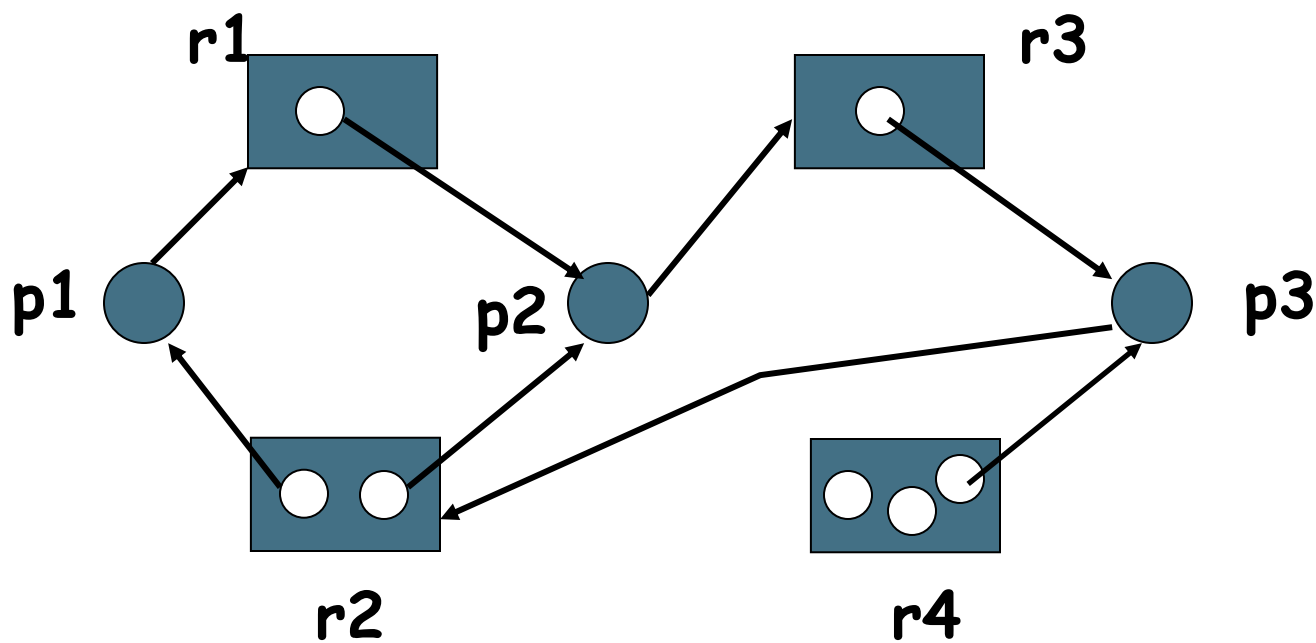
例1. $P = \{p1, p2, p3\}$, $R = \{r1(1), r2(2), r3(1), r4(3)\}$

$E = \{(p1, r1), (p2, r3), (r1, p2), (r2, p1), (r2, p2), (r3, p3), (r4, p3)\}$

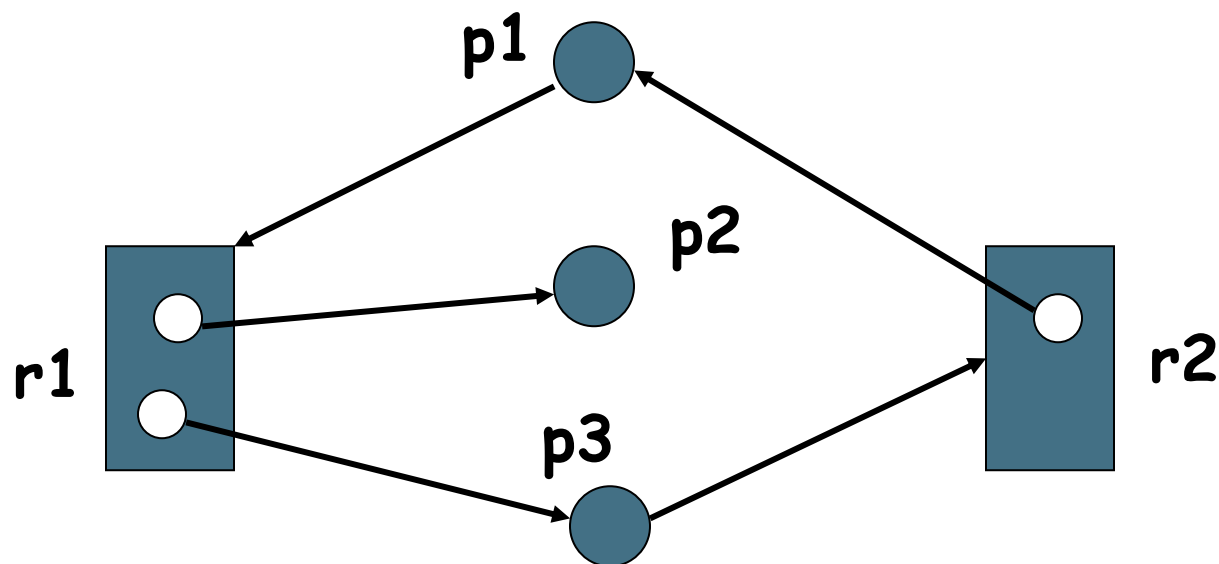


例子（有环路，有死锁）

增加边 (p3,r2)



例子（有环路，无死锁）



P2释放占有资源r1，分给P1， $(p1, r1)$ 改为 $(r1, p1)$

资源分配图的约简

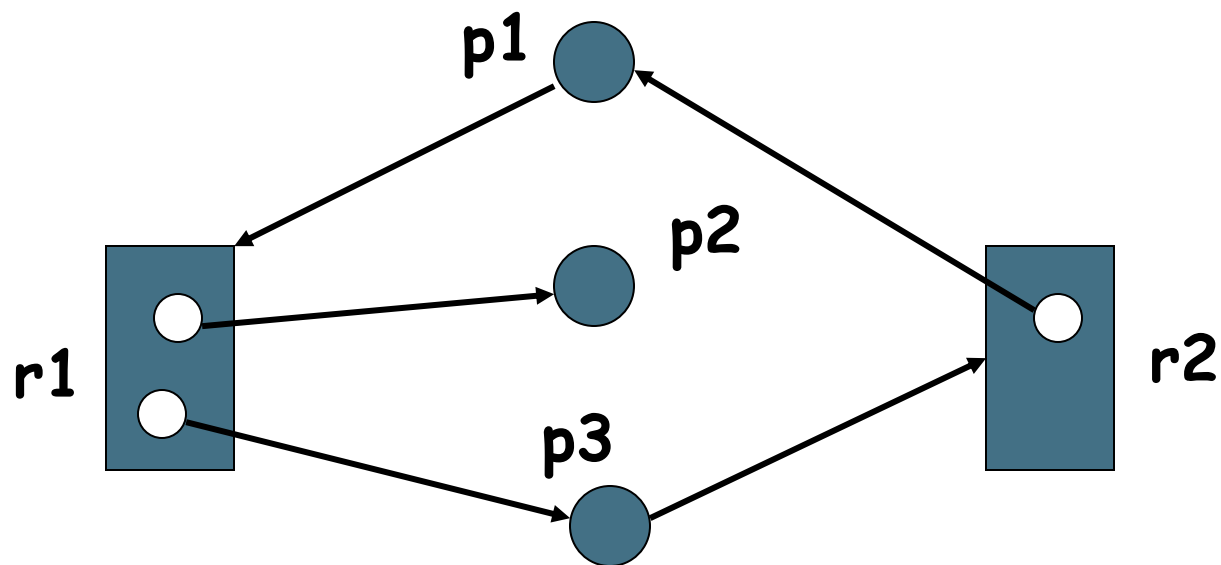
1. 寻找一个非孤立且没有请求边的节点 p_i ，若无算法结束；
2. 去除 p_i 的所有分配边使其成为一个孤立节点；
3. 寻找所有请求边都可满足的进程 p_j ，将 p_j 的所有请求边全部改为分配边；
4. 转步骤1.

若算法结束时, 所有节点均为孤点, 则称资源分配图是完全可约简的, 否则称为不可完全约简的.

死锁定理

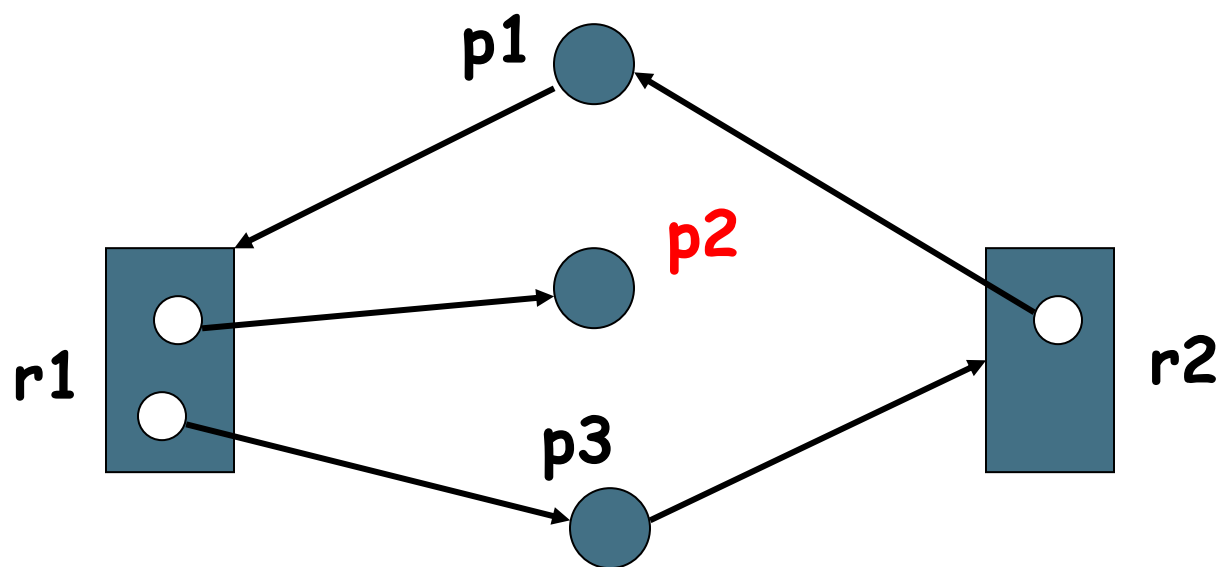
- 死锁定理
 - S为死锁状态的充分必要条件是S的资源分配图不可完全约简

例（有环路，无死锁）



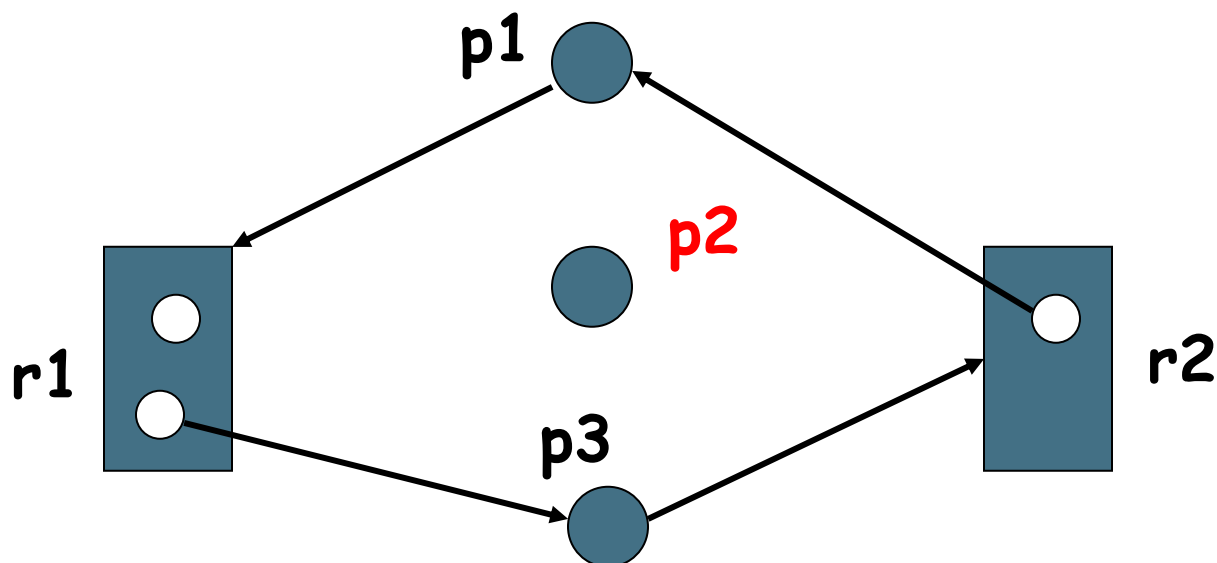
例（有环路，无死锁）

Step1 寻找一个非孤立且没有请求边的节点 (P2)



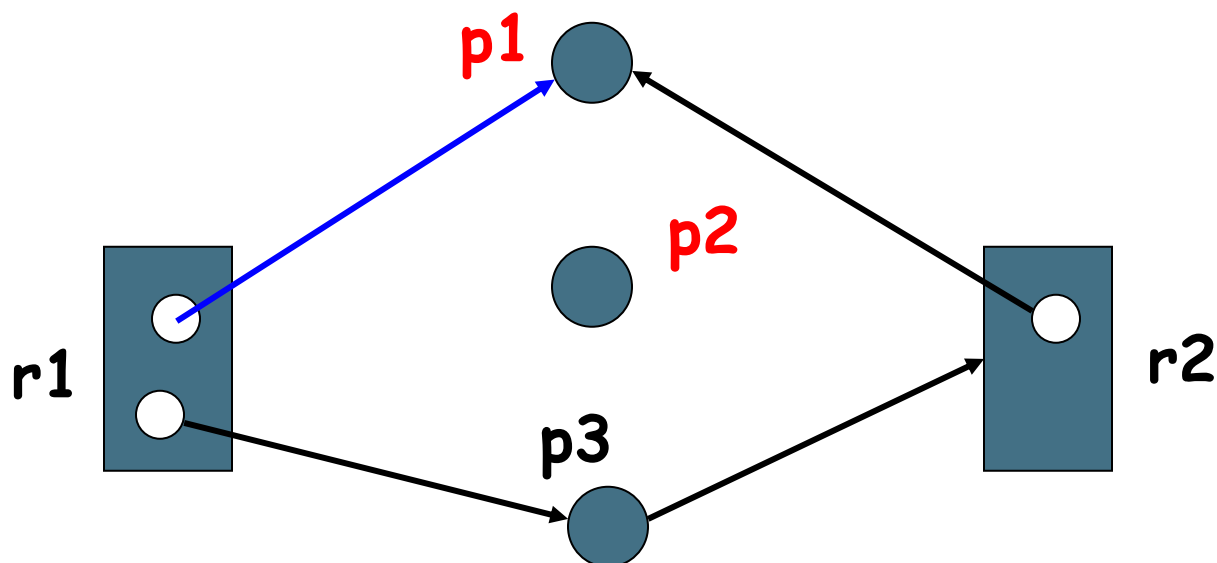
例（有环路，无死锁）

Step2去除P2所有分配边使其成为一个孤立节点



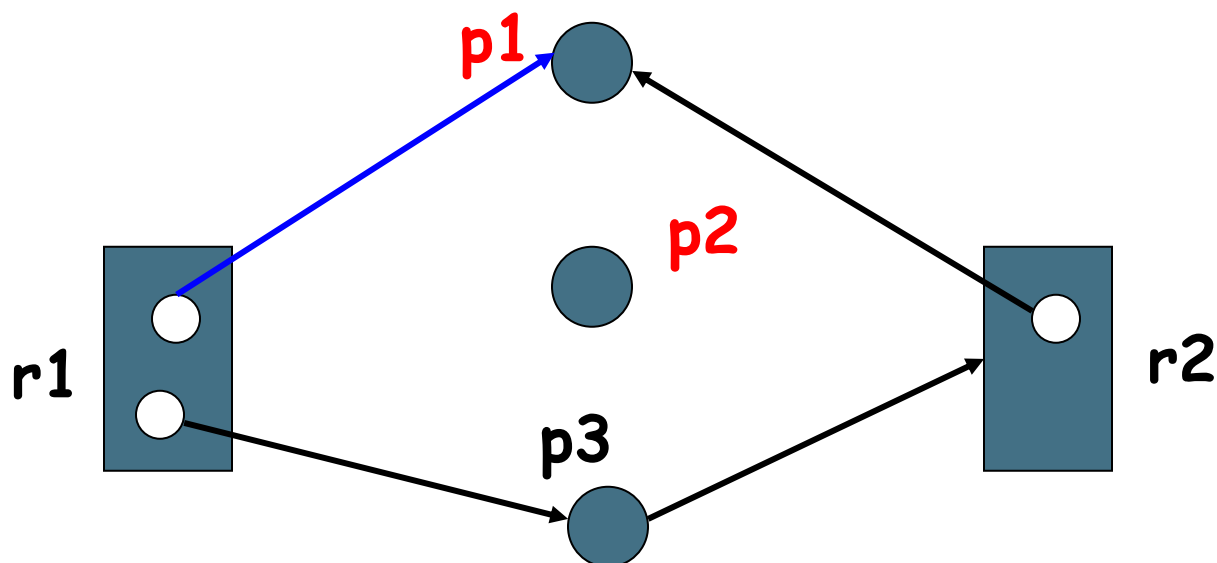
例（有环路，无死锁）

Step3 寻找所有请求边都可满足的进程p1，将p1的所有请求边全部改为分配边；



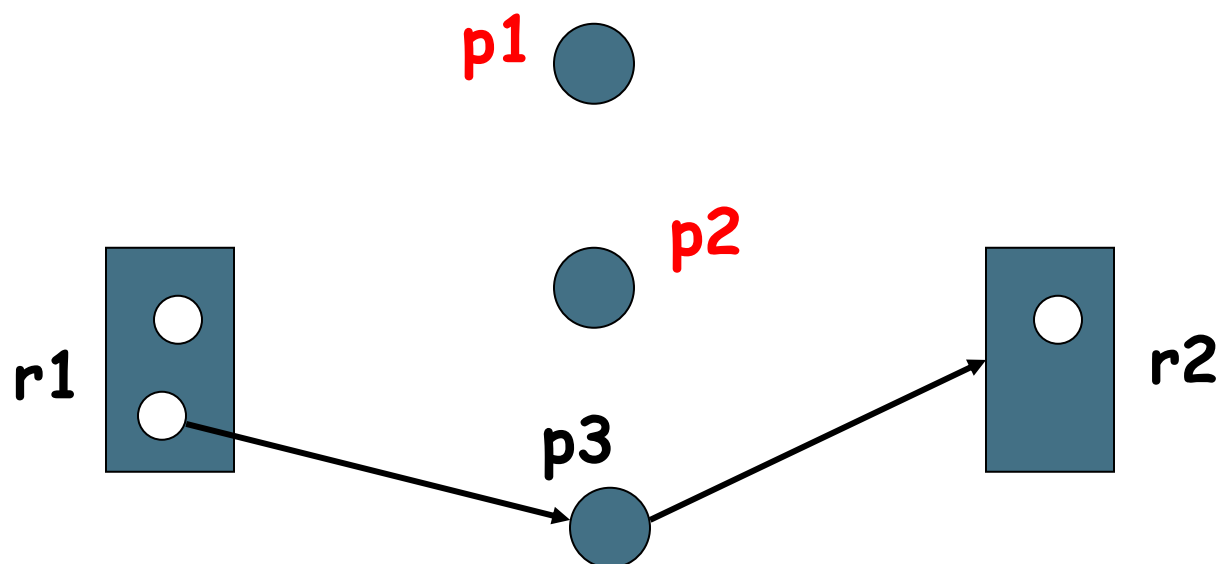
例（有环路，无死锁）

Step4 转步骤1（寻找一个非孤立且没有请求边的节点P1）



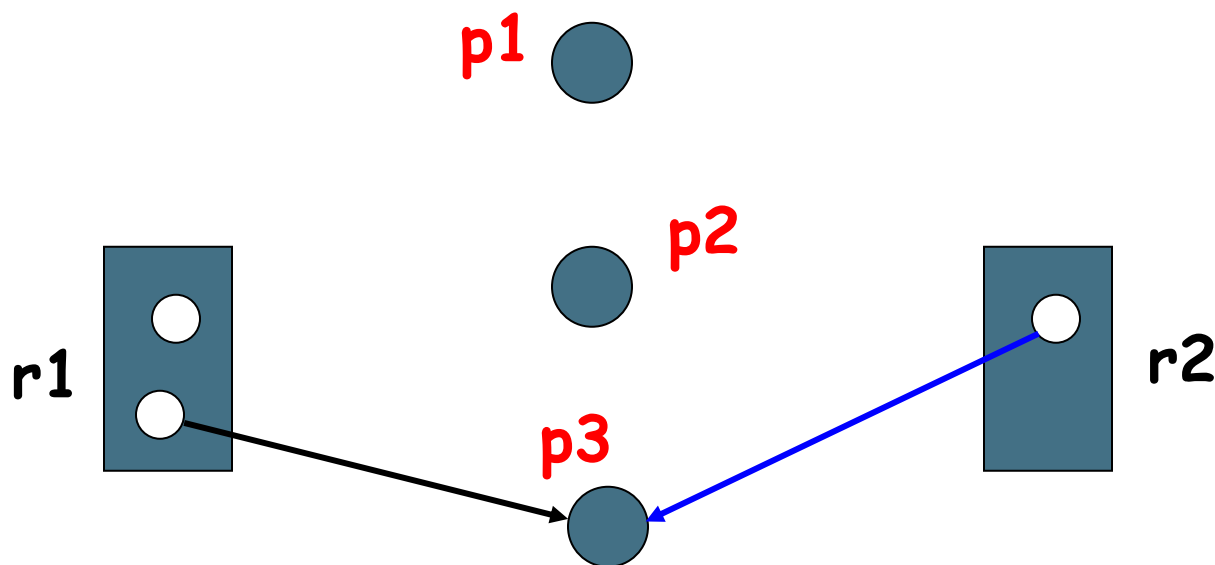
例（有环路，无死锁）

Step5 去除P1的所有分配边使其成为一个孤立节点



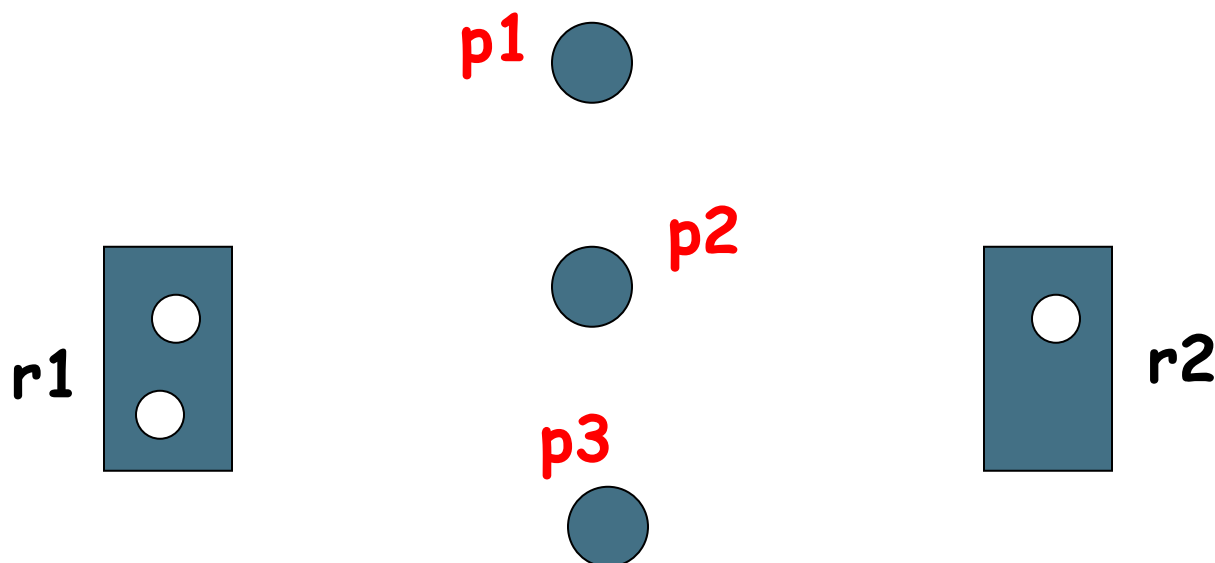
例（有环路，无死锁）

Step6 寻找所有请求边都可满足的进程p3，将p3的所有请求边全部改为分配边；

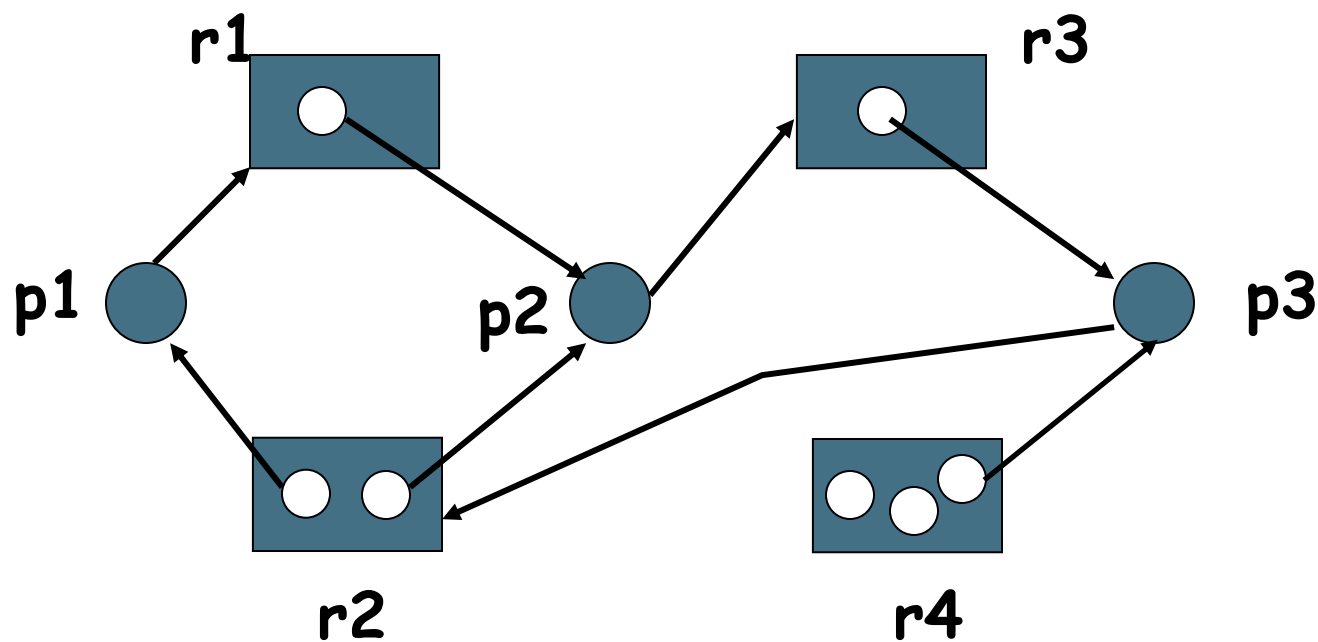


例（有环路，无死锁）

Step7 去除P3的所有分配边使其成为一个孤立节点, 此时, 所有节点均为孤点, 资源分配图可完全可约简, 故无死锁。



例（有环路，有死锁）



提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.5 死锁的处理

- 死锁预防 (Deadlock Prevention)
- 死锁避免 (Deadlock Avoidance)
- 死锁检测 (Deadlock Detection)
- 死锁恢复 (Deadlock Recovery)

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.6 死锁预防

- 对进程有关资源的申请活动加限制，所有进程遵循这种限制，即可保证没有死锁发生
 - 优点：简单，系统不需要做什么
 - 缺点：对进程约束，违反约束仍可能死锁
- 预防方法
 - 预先分配法
 - 有序分配法

5.6.1 预先分配法

- 进程：运行前申请所需全部资源
- 系统
 - 能够满足，全部分配
 - 否则，一个也不分配
- 由于进程在投入运行前已经占有所需要的全部资源，因而在运行期间不会再发出新的资源申请命令
- 破坏 “**hold-and-wait**” 条件

5.6.1 预先分配法(Cont.)

- 缺点：
 - 资源利用效率低
 - 一次提出申请困难

5.6.2 有序分配法

资源集： $R = \{r_1, r_2, \dots, r_n\}$ ，事先将所有资源类排序

函数： $F: R \rightarrow N$ 每个资源类赋予唯一的整数

规定进程必须按照资源编号从小到大的次序申请资源

例如： $R = \{\text{scanner}, \text{tape}, \text{printer}\}$

$F(\text{scanner})=1; F(\text{tape})=2; F(\text{printer})=3;$



进程 p_i 可以申请资源 r_j 中的实例 $\Leftrightarrow \forall r_l, p_i$ 当前已占有 $r_l, F(r_l) < F(r_j)$

5.6.2 有序分配法(Cont.)

- 一个进程可以申请某一资源类 r_i 中的资源实例的充要条件是它已经释放了资源类 r_j 中的所有资源实例，这里 $F(r_i) \leq F(r_j)$
- 如果遵从这个条件，就不会发生死锁
- 如果进程需要同一资源类中的若干个资源实例，则它必须在一个命令中同时发出请求

5.6.2 有序分配法(Cont.)

证明无死锁 (deadlock free) :

方法：反证，假定死锁

时刻 t_1 : p_1 无限等待 r_{k_1} 中的资源实例，被 p_2 占有；

t_2 : p_2 无限等待 r_{k_2} 中的资源实例，被 p_3 占有；

...

t_n : p_n 无限等待 r_{k_n} 中的资源实例，被 p_1 占有；

根据有序申请假设：

$$F(r_{k_1}) < F(r_{k_2}) < \dots < F(r_{k_n}) < F(r_{k_1})$$

矛盾。

有序分配策略通过破坏循环等待条件来预防死锁的发生。

5.6.2 有序分配法(Cont.)

- 优点

与预先分配相比, 资源利用率提高.

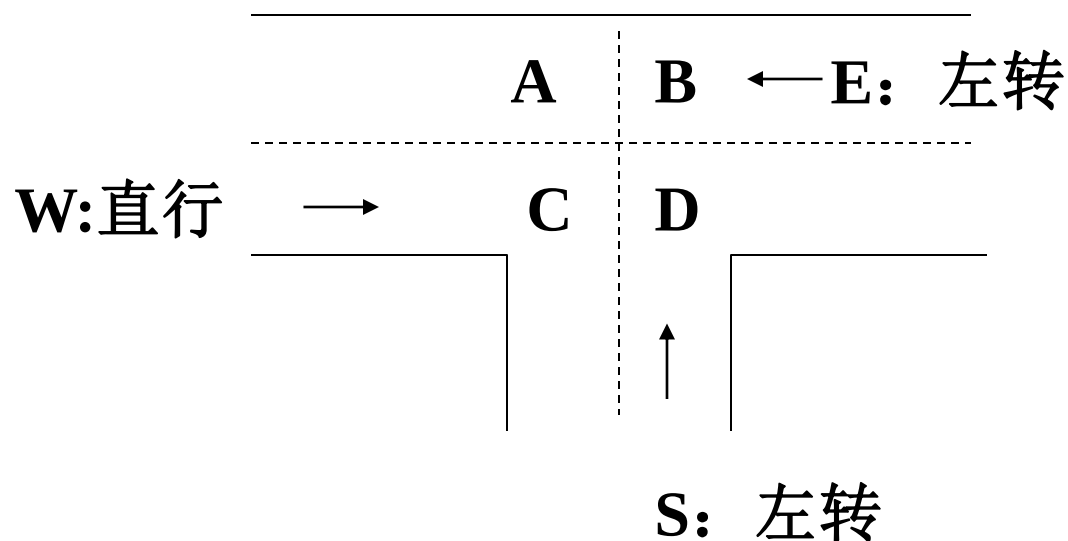
- 缺点

资源编号困难;

按序号申请资源增加了资源使用者负担, 一旦不按序号申请还是可能出现死锁;

为保持按序申请, 某些暂时不用的资源也需提前申请, 牺牲资源利用率.

例子



死锁的可能性：情形1，情形2

资源编号：F(D)=1；F(B)=2；F(A)=3；F(C)=4；

Var S1, S2, S3, S4: semaphore; (1, 1, 1, 1)

例子

Procedure S:

P(S1);

驶入D;

P(S2);

驶入B;

V(S1);

P(S3);

驶入A;

V(S2);

驶出A;

V(S3)

Procedure E:

P(S2);

驶入B;

P(S3);

驶入A;

V(S2);

P(S4);

驶入C;

V(S3);

驶出C;

V(S4);

Procedure W:

P(S1);

P(S4);

驶入C;

驶入D;

V(S4);

驶出D;

V(S1);

例子

COBEGIN

S1:S; ... ; Sm:S;

E1:E; ... ; En:E;

W1:W; ... ; Wo:W

COEND;

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

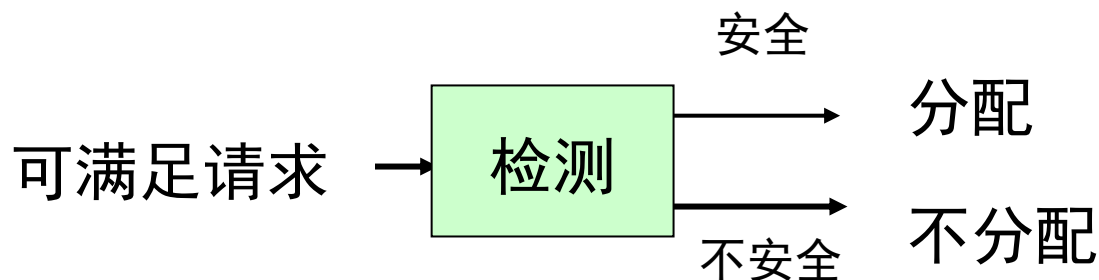
5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.7 死锁避免

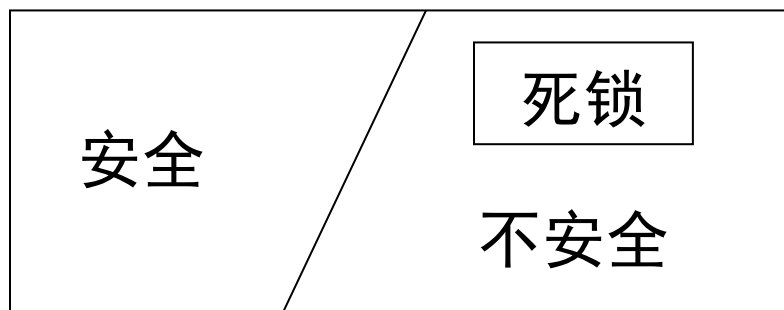
- 死锁避免

- 保证系统不进入死锁状态的动态策略
- 对于进程发出的每一个系统能够满足的资源申请命令实施动态检查，并根据检查结果决定是否实施资源分配



5.7 死锁避免(Cont.)

定义 系统处于安全状态：如果存在一个由系统中的所有进程构成的安全进程序列 $\langle p_1, p_2, \dots, p_n \rangle$ ，那么系统处于安全状态。
进程序列 $\langle p_1, p_2, \dots, p_n \rangle$ 安全， $p_1, p_2, \dots, p_n$ 可依次进行完。



对进程序列 $\langle p_1, p_2, \dots, p_n \rangle$ ，如果有：

p_i 要申请的资源数 $\leq$ 当前剩余资源数 $+$ $\sum_{j < i} p_j$ （ p_j 当前占有资源数），则称该进程序列是安全的。

银行家算法 (Cont.)

Banker's algorithm, E. W. Dijkstra.

进程：事先申明所需资源最大量（并不分配）

$Claim = Max$

系统：对每个可满足的资源申请命令进行安全性检查。

安全, 分配; 不安全: 不分配

$P = \{p_1, p_2, \dots, p_n\};$

$R = \{r_1, r_2, \dots, r_m\};$

银行家算法 (Cont.)

数据结构:

Available: array[1..m] of integer; //系统可用资源
Claim: array[1..n, 1..m] of integer; //进程最大需求
Allocation: array[1..n, 1..m] of integer; //当前分配
Need: array[1..n, 1..m] of integer; //尚需资源
Request: array[1..n, 1..m] of integer; //当前请求

临时变量:

Work: array[1..m] of integer; //记录可用资源
Finish: array[1..n] of boolean; //记录进程是否可以执行完

银行家算法 (Cont.)

设 X, Y 为下标 $1 \dots l$ 的一维数组:

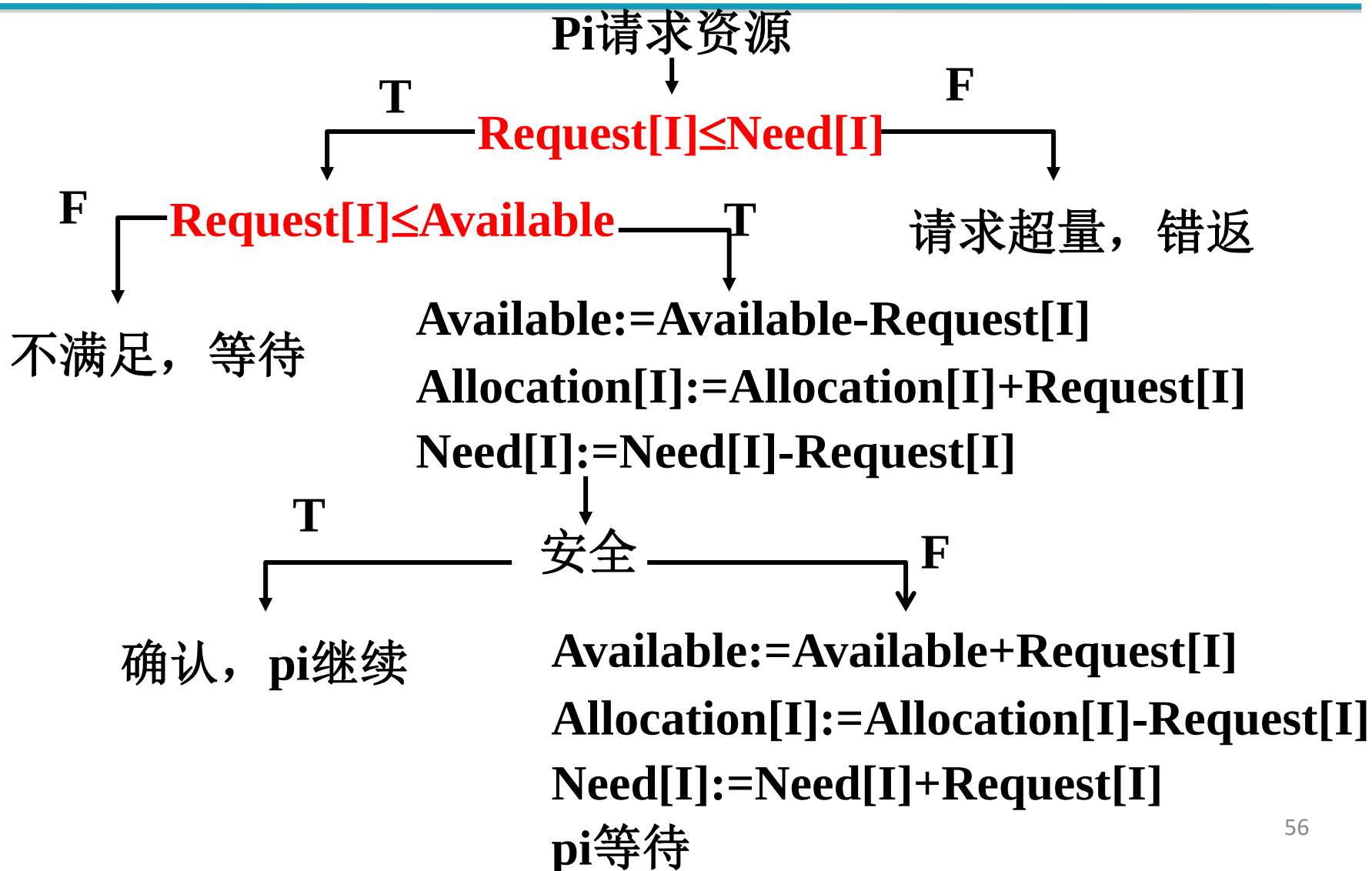
$$X \leq Y \Leftrightarrow \forall j \ (1 \leq j \leq l), \ X[j] \leq Y[j]$$

$$X := Y \Leftrightarrow \forall j \ (1 \leq j \leq l), \ X[j] := Y[j]$$

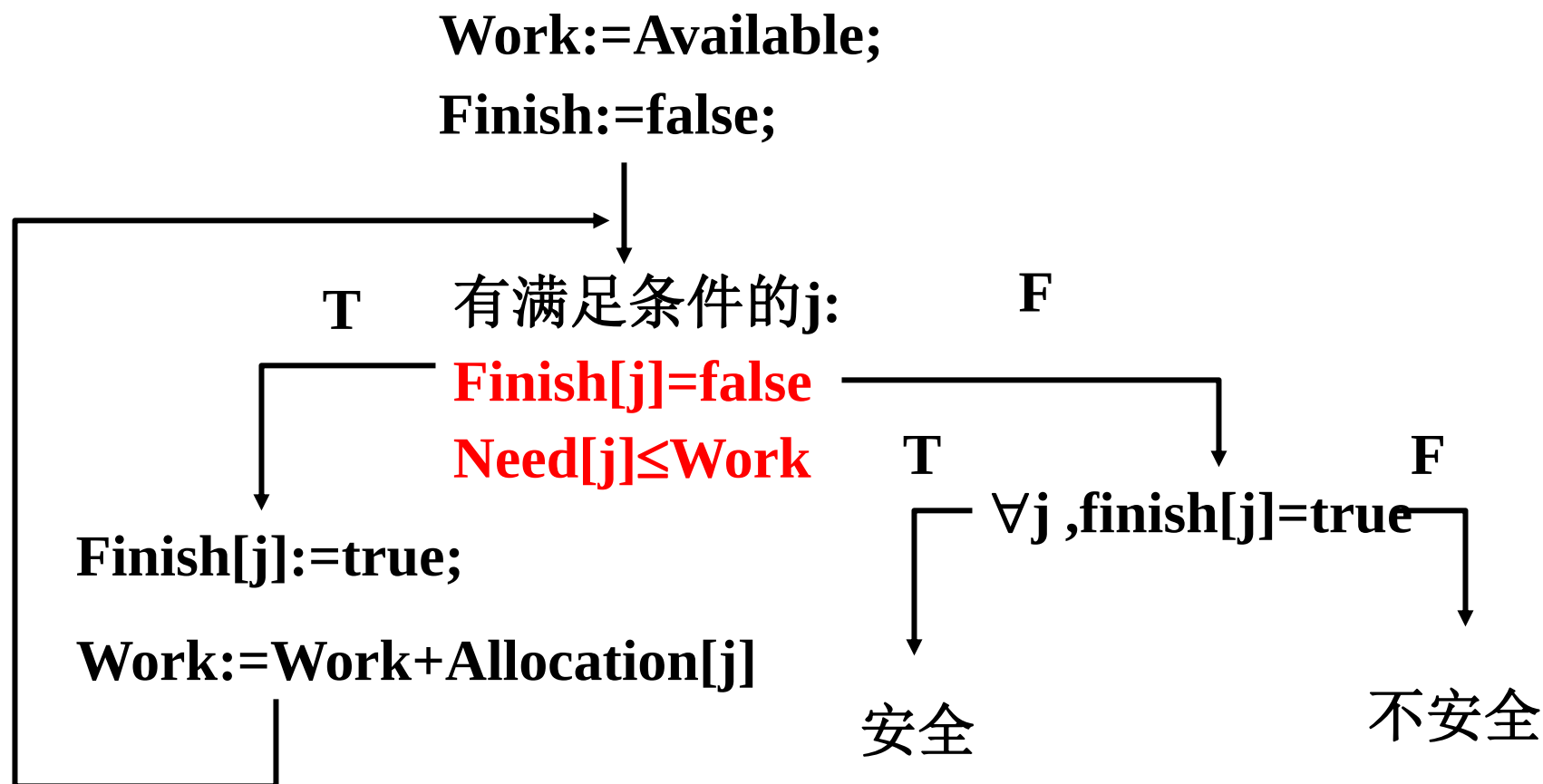
$$X := c \Leftrightarrow \forall j \ (1 \leq j \leq l), \ X[j] := c$$

$$X \pm Y \Leftrightarrow \forall j \ (1 \leq j \leq l), \ X[j] \pm Y[j]$$

资源分配算法



安全性检测算法



银行家算法例子

$R=\{A(10),B(5),C(7)\}$ $P=\{p0,p1,p2,p3,p4\}$

	<u>Claim</u>			<u>Allocation</u>			<u>Need</u>			<u>Available</u>				<u>Work</u>			<u>Finish</u>
	A	B	C	A	B	C	A	B	C	A	B	C		A	B	C	
P0:	7	5	3	0	1	1	7	4	2	3	3	3					
p1:	3	2	2	2	0	0	1	2	2								
p2:	9	0	2	3	0	0	6	0	2								
p3:	2	2	2	2	1	1	0	1	1								
p4:	4	3	3	0	0	2	4	3	1								

安全进程序列: $\langle p1,p3,p4,p2,p0 \rangle$

p2请求: $Request[2]=(1,0,2)$

银行家算法例子 (Cont.)

假定分配:

	<u>Claim</u>			<u>Allocation</u>			<u>Need</u>			<u>Available</u>			<u>Work</u>		<u>Finish</u>
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C
P0:	7	5	3	0	1	1	7	4	2	2	3	1			
p1:	3	2	2	2	0	0	1	2	2						
p2:	9	0	2	4	0	2	5	0	0						
p3:	2	2	2	2	1	1	0	1	1						
p4:	4	3	3	0	0	2	4	3	1						

安全进程序列: <p3,p4,p1,p2,p0>, P2请求可分配

p4请求: Request[4]=(3,3,0), 是否分配?

不能满足, P4等待

银行家算法例子 (Cont.)

假定分配:

	<u>Claim</u>			<u>Allocation</u>			<u>Need</u>			<u>Available</u>			<u>Work</u>			<u>Finish</u>		
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C			
P0:	7	5	3	0	1	1	7	4	2	2	3	1						
p1:	3	2	2	2	0	0	1	2	2									
p2:	9	0	2	4	0	2	5	0	0									
p3:	2	2	2	2	1	1	0	1	1									
p4:	4	3	3	0	0	2	4	3	1									

p0请求: Request[0]=(0,2,1), 是否分配?

银行家算法例子 (Cont.)

假定分配:

	<u>Claim</u>			<u>Allocation</u>			<u>Need</u>			<u>Available</u>			<u>Work</u>			<u>Finish</u>
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C	
P0:	7	5	3	0	3	2	7	2	1	2	1	0				
p1:	3	2	2	2	0	0	1	2	2							
p2:	9	0	2	4	0	2	5	0	0							
p3:	2	2	2	2	1	1	0	1	1							
p4:	4	3	3	0	0	2	4	3	1							

不安全，取消分配，P0等待。

银行家算法的保守性

例子： $R=\{A(1),B(1)\}$, 申请 a, b ; 释放 $\bar{a}, \bar{b}$

$P=\{p1,p2\}$, $p1: a\ b\ \bar{a}\ \bar{b}$; $p2: b\ \bar{b}\ b\ a\ \bar{a}\ \bar{b}$

某时刻系统状态如下：

	<u>Claim</u>		<u>Allocation</u>		<u>Need</u>		<u>Available</u>			<u>Work</u>		<u>Finish</u>
	A	B	A	B	A	B	A	B		A	B	
p1:	1	1	0	0	1	1	1	1				
p2:	1	1	0	0	1	1						

$Request[1]=(1,0)$, 是否分配？

银行家算法的保守性

分配后：

	<u>Claim</u>		<u>Allocation</u>		<u>Need</u>		<u>Available</u>		<u>Work</u>	<u>Finish</u>
	A	B	A	B	A	B	A	B	A	B
p1:	1	1	1	0	0	1	0	1		
p2:	1	1	0	0	1	1				

安全，分配。

银行家算法的保守性 (Cont.)

分配后:

	<u>Claim</u>		<u>Allocation</u>		<u>Need</u>		<u>Available</u>		<u>Work</u>	<u>Finish</u>
	A	B	A	B	A	B	A	B	A	B
p1:	1	1	1	0	0	1	0	1		
p2:	1	1	0	0	1	1				

- 如继续Request[2]=(0,1), 是否分配?

银行家算法的保守性 (Cont.)

分配后:

	<u>Claim</u>		<u>Allocation</u>		<u>Need</u>		<u>Available</u>		<u>Work</u>	<u>Finish</u>
	A	B	A	B	A	B	A	B	A	B
p1:	1	1	1	0	0	1	0	0		
p2:	1	1	0	1	1	0				

- 如继续Request[2]=(0,1), 不安全, 不分配, P2请求等待

银行家算法的保守性 (Cont.)

例子: $R=\{A(1),B(1)\}$, 申请 a, b ; 释放 $\bar{a}, \bar{b}$

$P=\{p1,p2\}$, $p1: a \ b \ \bar{a} \ \bar{b}$; $p2: b \ \bar{b} \ b \ a \ \bar{a} \ \bar{b}$

$Request[1]=(1,0)$, $Request[2]=(0,1)$,

- 实际上, 如果真正实施资源分配, 系统并不会处于死锁状态。因为分配资源后按照 $P2(\bar{b}), P1(b), P1(\bar{a}), P1(\bar{b}), P2(b), P2(a), P2(\bar{a}), P2(\bar{b})$ 的次序, 两个进程都可以执行完毕

死锁状态是不安全状态的真子集

Remarks

- 银行家算法要求条件：进程所需资源最大量，这个信息对于充要性分析是不够的
- 假设：进程预先给出有关资源的命令序列，则可以给出死锁避免的充要性算法
- 预先给出进程有关资源的命令序列是困难的（程序的分枝和循环）

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.8 死锁的发现

- 死锁发现：如果一个系统中既未采用死锁预防策略，也未采用死锁避免策略，则系统有可能发生死锁
- 死锁检测算法：由Shoshani和Coffman提出

5.8.1 死锁检测算法

数据结构:

Available: array[1..m] of integer;

Allocation: array[1..n, 1..m] of integer;

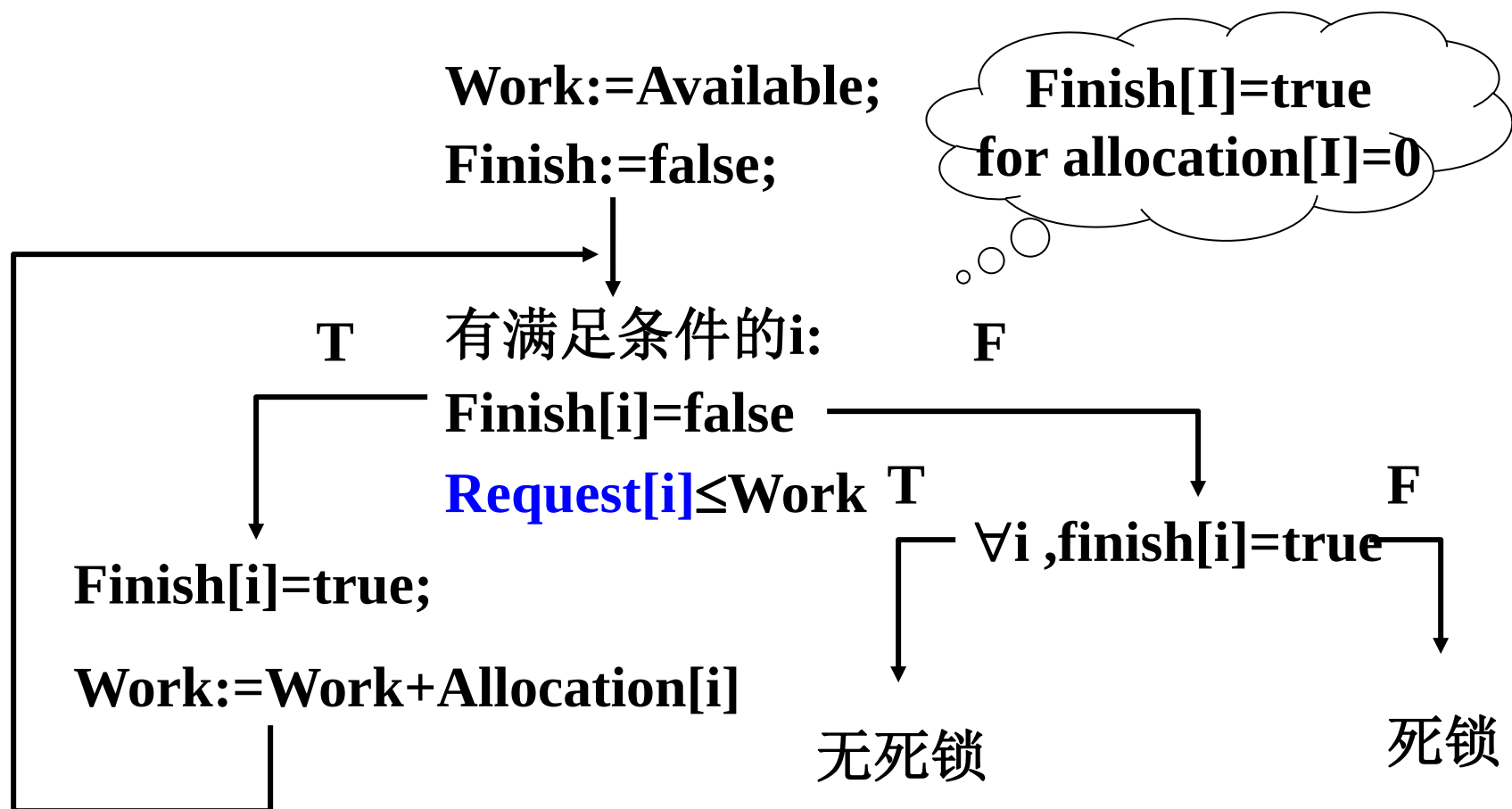
Request: array[1..n, 1..m] of integer;

临时变量:

Work: array[1..m] of integer;

Finish: array[1..n] of Boolean;

5.8.1 死锁检测算法 (Cont.)



Remarks

- 上述算法可以检测到参与死锁的全部进程
- 只检测占有资源的进程，初始化时：

$\text{Finish}[i]=\text{true}$, for $\text{Allocation}[i]=0$

Remarks (Cont.)

- 令 P 是所有进程的集合， P' 包含于 P ，是所有占有资源的进程集合，则：
 - $P \text{死锁} \Leftrightarrow P' \text{死锁}$
- 死锁检测之后是恢复，只考虑 P' 中的进程
- 死锁检测不考虑 $P-P'$ 中的进程，缩小了检测范围，降低了系统开销

死锁例子

例子： $R=\{A(7),B(3),C(6)\}$; $P=\{p_0,p_1,p_2,p_3,p_4\}$

<u>Allocation</u>	<u>Request</u>	<u>Available</u>	<u>Work</u>	<u>Finish</u>
A B C	A B C	A B C	A B C	
p0: 0 1 0	0 0 0	0 1 0		
p1: 2 0 0	2 0 2			
p2: 3 0 3	0 0 0			
p3: 2 1 1	1 0 0			
p4: 0 0 2	0 0 2			

存在安全序列 $\langle p_0,p_2,p_3,p_4,p_1 \rangle$,故未死锁。

死锁例子

例子： $R=\{A(7),B(3),C(6)\}$; $P=\{p_0,p_1,p_2,p_3,p_4\}$

<u>Allocation</u>	<u>Request</u>	<u>Available</u>	<u>Work</u>	<u>Finish</u>
A B C	A B C	A B C	A B C	
p0: 0 1 0	0 0 0	0 1 0		
p1: 2 0 0	2 0 2			
p2: 3 0 3	0 0 0			
p3: 2 1 1	1 0 0			
p4: 0 0 2	0 0 2			

如果 $Request[2]=(0,0,1)$, 是否死锁?

死锁例子

例子： $R=\{A(7),B(3),C(6)\}$; $P=\{p0,p1,p2,p3,p4\}$

<u>Allocation</u>	<u>Request</u>	<u>Available</u>	<u>Work</u>	<u>Finish</u>
A B C	A B C	A B C	A B C	
p0: 0 1 0	0 0 0	0 1 0		
p1: 2 0 0	2 0 2			
p2: 3 0 3	0 0 1			
p3: 2 1 1	1 0 0			
p4: 0 0 2	0 0 2			

如果 $Request[2]=(0,0,1)$, 死锁, 参与死锁进程 $\{p1,p2,p3,p4\}$

5.8.2 死锁检测时刻

- 考虑因素
 - 死锁发生频度
 - 死锁影响进程数
- 检测时间
 - 等待时检测
 - 定时检测
 - 资源 (eg. CPU) 利用率下降时检测
 - 交互式任务没有响应时检测

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.9 死锁的恢复

1. 重新启动：是最简单、最常用的死锁消除方法，但代价很大。
2. 终止进程(process termination)
 - (1) 一次性全部终止；
 - (2) 逐步终止(优先级，代价函数)
3. 剥夺资源(resource preemption)
 - (1) 逐步剥夺
 - (2) 一次剥夺
4. 进程回退(rollback): 让参与死锁的进程回退到以前没有发生死锁的某个点处，并由此点开始继续执行，希望进程交叉执行时不再发生死锁。但是系统开销很大

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.10 鸵鸟算法

- 视而不见

- Pro:

- 工程师观点 (考虑死锁发生的频率, 危害, 处理代价)
 - 死锁发生频率 $\ll$ 其它故障引起的系统瘫痪的频率
 - 死锁处理 constant overhead $\gg$ 危害
 - Unix和Windows等商用系统都采用鸵鸟算法
 - 一般用户宁愿忍受系统的偶然性故障带来的损失, 也不愿意经常进行死锁处理而牺牲系统性能

- Cont:

- 数学家观点
 - 必须处理, 无论代价如何

- 目前系统实际如此

- Eg. UNIX proc结构(100)、交换机制

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.11 有关问题的讨论

- 关于充要性算法

- 死锁预防

- 无开销，但使用不便，资源利用率低，而且若进程违反相关协议仍然可能发生死锁

- 死锁避免

- 开销大，且不是必要性算法
 - 充要性算法需要比银行家算法还强的条件
 - 银行家算法只给出了进程需要资源的最大量，而所需要资源的具体申请和释放顺序仍然是未知的，因而银行家算法只能往最坏处设想

5.11 有关问题的讨论(Cont.)

- 关于消耗型资源问题

- 消耗型资源

- 又称生灭资源，是指系统运行过程中动态产生、动态消亡的资源，如消息
 - 当发送者执行send系统调用命令便产生了这种资源
 - 当接收者执行receive系统调用时便消耗了这种类型的资源

- 解决死锁策略

- 超时技术

5.11 有关问题的讨论(Cont.)

- 关于可剥夺资源问题

- 处理器

- 一个处理器在一段时间内只能分配给一个进程，当出现高优先级别进程允许抢先
 - 多个进程不会因为竞争处理器而发生死锁

- 内存

- 当占有一部分内存的进程申请新内存资源无法满足时，为其分配外存空间
 - 如果外存空间也不能满足，死锁可能发生
 - 但一般很少见，因为外存空间很大，这是虚拟存储的优点

5.11 有关问题的讨论(Cont.)

- 关于两阶段锁
 - 两阶段封锁协议： 增长阶段（加锁）和消退阶段（开锁）；
 - 增长阶段有可能发生死锁。

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.12 饥饿与活锁

- 饥饿 vs 死锁
 - 死锁进程处于等待状态，忙式等待的进程并非处于等待状态，但却可能被饿死
 - 死锁进程等待永远不会释放的资源，饿死进程等待可能被释放，但却不会分给自己的资源，其等待时间没有上界
 - 死锁一定发生了循环等待，饿死不然
 - 死锁至少涉及两个进程，饿死进程可能只有一个

5.12 饥饿与活锁

- 饥饿 (starvation)
 - 当等待时间给进程推进和响应带来明显影响时, 称发生了进程饥饿
 - 饥饿到一定程度的进程所赋予的使命即使完成也不再具有实际意义时称该进程被饿死
- 没有时间上界的等待
 - 排队等待
 - 忙式等待
- 忙式等待条件下发生的饥饿, 称为活锁 (live lock)

活锁

- 排队等待：当进程申请资源不能满足时，便进入等待状态，待资源得到时被唤醒，这种等待是排队等待。
- 排队等待与忙式等待
 - 排队等待的进程不会分配处理机，因而减少对处理机不必要的占用时间
 - 排队等待伴随一定的系统开销：运行→等待，等待→就绪，就绪→运行，经过两次进程切换
 - 在资源竞争不是很激烈的情况下，进程等待的时间比较短，2次进程切换的时间大于等待时间，可以让进程忙等
- 忙式等待条件下发生的饥饿,称为**活锁(live lock)**.
 - 活锁与死锁不同，活锁可能会解除

活锁

- 例如：在UNIX v6中，进程表proc结构共有50个表项，即系统最多可以同时建立50个进程，假设当前有10个进程要求创建子进程，且每个进程欲创建5个子进程，如创建失败则睡眠一段时间再继续创建，即执行如下代码

```
for(i=1;i<=5;i++)  
{  
    while(fork()==-1)//不成功  
        sleep(5);  
    {...}  
}
```

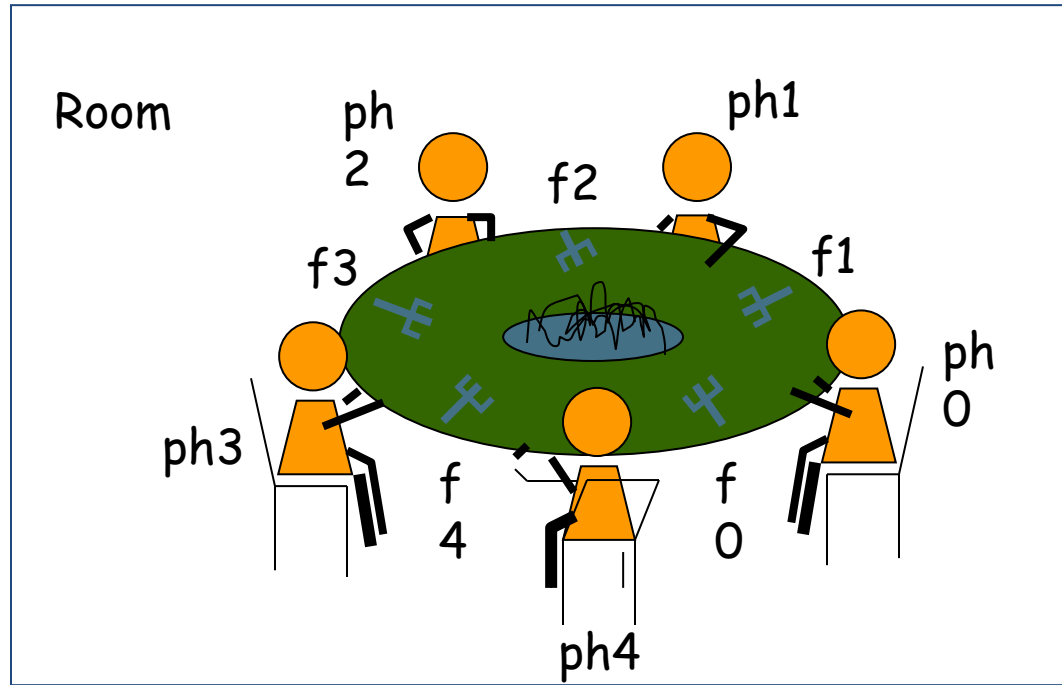
当每个进程创建了4个子进程时，proc表用完，则进程进入忙式等待，若此时等待时间有影响则进入活锁。但是如果某些被创建的进程结束，proc空闲，忙等的进程可能结束忙等。

例3. 哲学家就餐问题(Cont.)

■ 问题描述

- 有一个房间，其中有5位哲学家，他们的生活就是思考和进食。房间中摆着一张圆桌，就餐的桌子，圆桌的中间放着一个盘子，盘子中装有通心粉，这是哲学家进食时所吃的食物。这里假定盘子中的食物是无限多的，哲学家永远吃不完
- 有5个哲学家，哲学家在就餐时坐在自己的座位上(位置不能乱串)。当哲学家思考时，他可以离开自己的座位在房间内走动，但是当他想要进食时，则必须做到自己的座位上。在哲学家就餐时，需要一个工具，这个工具就是叉子
- 在圆桌上，每2个哲学家之间摆放一把叉子。当哲学家进食时需要使用其左边和右边的2把叉子
- 叉子不是同种组合资源，因为涉及到放的位置不同

例3. 哲学家就餐问题(Cont.)



例3. 哲学家就餐问题(Cont.)

哲学家活动:

Do{

 思考

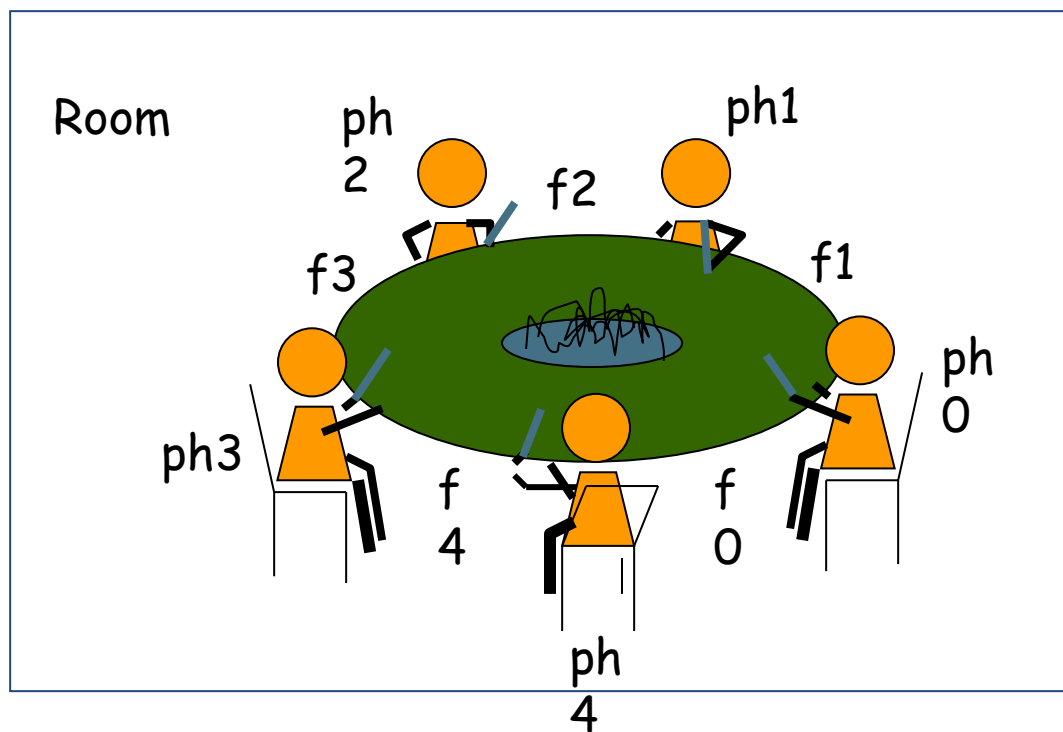
 进食

}While(1)

```
▪ semaphore fork [5] = {1}; /* 筷子*/
▪ inti;
▪ void philosopher (inti)
▪ {
▪ while (true) {
▪     think();
▪     P (fork[i]);
▪     P (fork [(i+1) mod 5]);
▪     eat();
▪     V (fork [(i+1) mod 5]);
▪     V (fork[i]);
▪ }
▪ }
▪ void main()
▪ {
▪ parbegin(philosopher (0), philosopher (1),philosopher (2), philosopher (3),philosopher (4));
▪ /* 五个哲学家并发执行*/
▪ }
```

例3. 哲学家就餐问题(Cont.)

死锁情况：每位哲学家拿到左叉，等待右叉



为防止死锁发生可采取的措施

- 最多允许4个哲学家同时坐在桌子周围
- 仅当一个哲学家左右两边的叉子都可用时，才允许他拿筷子
- 给所有哲学家编号，奇数号的哲学家必须首先拿左边的筷子，偶数号的哲学家则反之

第一种解决方案——4个哲学家就餐

同时只允许4位哲学家就餐

```
var fork : array [0...4] of semaphore;  
    fork [0], fork [1], fork [2], fork [3], fork [4] :=1;  count:semaphore;  
count := 4;  
process PH ( i)                eat;  
begin                          V(fork[ i]);  
    repeat                    V(fork[(i+1) mod 5]);  
        think;                V( count);  
        P( count );            until false;  
        P (fork [i]);          end;  
        P(fork [(i+1)mod 5]);
```

第二种解决方案——管程

哲学家活动:

```
Do{  
    思考  
    进食  
}While(1)
```



哲学家活动（包含资源活动）

```
Do{  
    思考  
    取左叉，取右叉  
    进食  
    放左叉，放右叉  
}While(1)
```

进食：需要“叉子”

叉子：不同种组合资源

第二种解决方案——管程

哲学家活动:

Repeat

 思考

 L: 取左叉

 If 右叉空闲 Then

 Begin 取右叉, 进食, 放左叉, 放右叉 end;

 Else Begin 放左叉, goto L end;

Until false

第二种解决方案——管程

```
Type dining-philosophers=MONITOR
  Var fork:Array[0..4]Of(free,used);
    q:Array[0..4]Of condition;
  define pick_up, try_pick_up, put_down;
  Procedure pick_up(I:0..4);
    Begin
      If fork[I]=used Then
        wait(q[I]);
        fork[I]:=used
      End;
```

第二种解决方案——管程

```
Function try_pick_up(I:0..4):boolean;  
  Begin  
    If fork[I]=used Then  
      try_pick_up:=false  
    Else Begin  
      fork[I]:=used;  
      try_pick_up:=true  
    End  
  End;  
End;
```

第二种解决方案——管程

```
Procedure put_down(I:0..4);  
  Begin  
    fork[I]:=free;  
    signal(q[I])  
  End;  
Procedure init;  
  Var I:0..4;  
  Begin  
    For I:=0 to 4 Do fork[I]:=free;  
  End;  
Begin init End;
```

第二种解决方案——管程

```
Var forks:dining_philosophers;
```

```
I号哲学家活动:
```

```
  Repeat
```

```
    {thinking}
```

```
    with_two_forks:=false;
```

```
  Repeat
```

```
    forks.pick_up(I);
```

```
    If forks.try_pick_up((I+1)mod 5)Then
```

```
      with_two_forks:=true
```

```
    Else forks.put_down(I)
```

```
  Until with_two_forks; //为true跳出循环
```


第二种解决方案——管程

```
{Eating}  
forks.put_down(I);  
forks.put_down((I+1)mod 5)  
Until false;
```

第三种解决方案——PV操作

死锁预防策略

死锁解决方法：同时申请左、右两把叉子(预先分配)。

哲学家状态描述(增加了hungry状态)：仅在饥饿的时候才申请叉子，且同时申请左、右两把叉子

State: Array[0..4]Of (thinking, hungry, eating);

Self: Array[0..4]Of semaphore; (initial value is 0);

就餐条件测试

测试I号哲学家是否具备进食条件：

```
Procedure test (I:0..4)
```

```
  If (state[I]=hungry) and  
    (state[(I-1)mod 5]<>eating) and  
    (state[(I+1)mod 5]<>eating)
```

```
  Then state[I]:=eating; V(self[I])
```

Remark:

如果i号哲学家处于饥饿状态，且左右两边的哲学家都不处于进食状态，那么将i号哲学家状态改为进食状态，同时唤醒i号哲学家。

未考虑互斥问题

I号哲学家活动:

1. Repeat
2. 思考
3. $state[I] := hungry;$
4. $test(I);$
5. $P(self[I]);$
6. 取左叉, 取右叉;
7. 进食
8. 放左叉, 放右叉;
9. $state[I] := thinking;$
10. $test((I-1) \bmod 5);$
11. $test((I+1) \bmod 5)$
12. Until false;

共享变量: $state;$

临界区域: 3-4, 9-11

Var mutex: semaphore; (1)

考虑互斥问题 (Cont.)

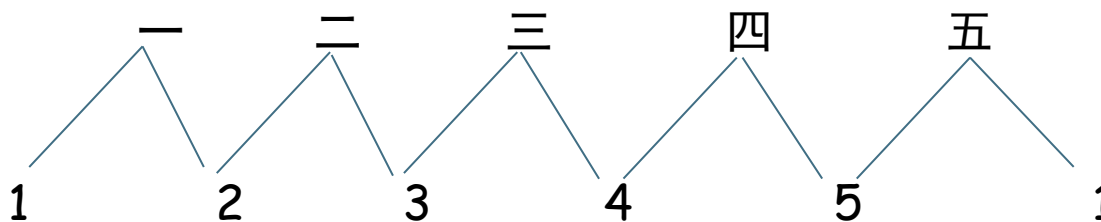
I号哲学家活动:

1. Repeat
2. 思考
3. P(mutex);
4. state[I]:=hungry;
5. test(I);
6. V(mutex);
7. P(self[I]);
8. 取左叉, 取右叉;
9. 进食
10. 放左叉, 放右叉;
11. P(mutex);
12. state[I]:=thinking;
13. test((I-1) mod 5);
14. test((I+1) mod 5);
15. V(mutex)
16. Until false;

问题

饿死情况：

某哲学家左右邻居至少有一个处于eating状态。



程序

```
Program dining_philosophers
var state:array[0..4]of (thinking,hungry,eating);
    self:array[0..4]of semaphore;
    mutex:semaphore;
procedure test(I:0..4)
begin
    if (state[I]=hungry)and
        (state[(I-1)mod 5]<>eating)
        and(state[(I+1)mod 5]<>eating) then
        begin state[I]:=eating; V(self[I]) end
end;
```

程序(Cont.)

```
Procedure philosopher(I:0..4)
begin
  cycle
    {thinking}
    P(mutex);
    state[I]:=hungry;
    test(I);
    V(mutex);
    P(self[I]);
```


程序(Cont.)

```
pick_up_fork(I);pick_up_fork((I+1)mod 5);  
{Eating};  
put_down_fork(I);put_down_fork((I+1)mod 5);  
P(mutex);  
state[I]:=thinking;  
test((I-1)mod 5);  
test((I+1)mod 5);  
V(mutex);  
end  
end
```

程序(Cont.)

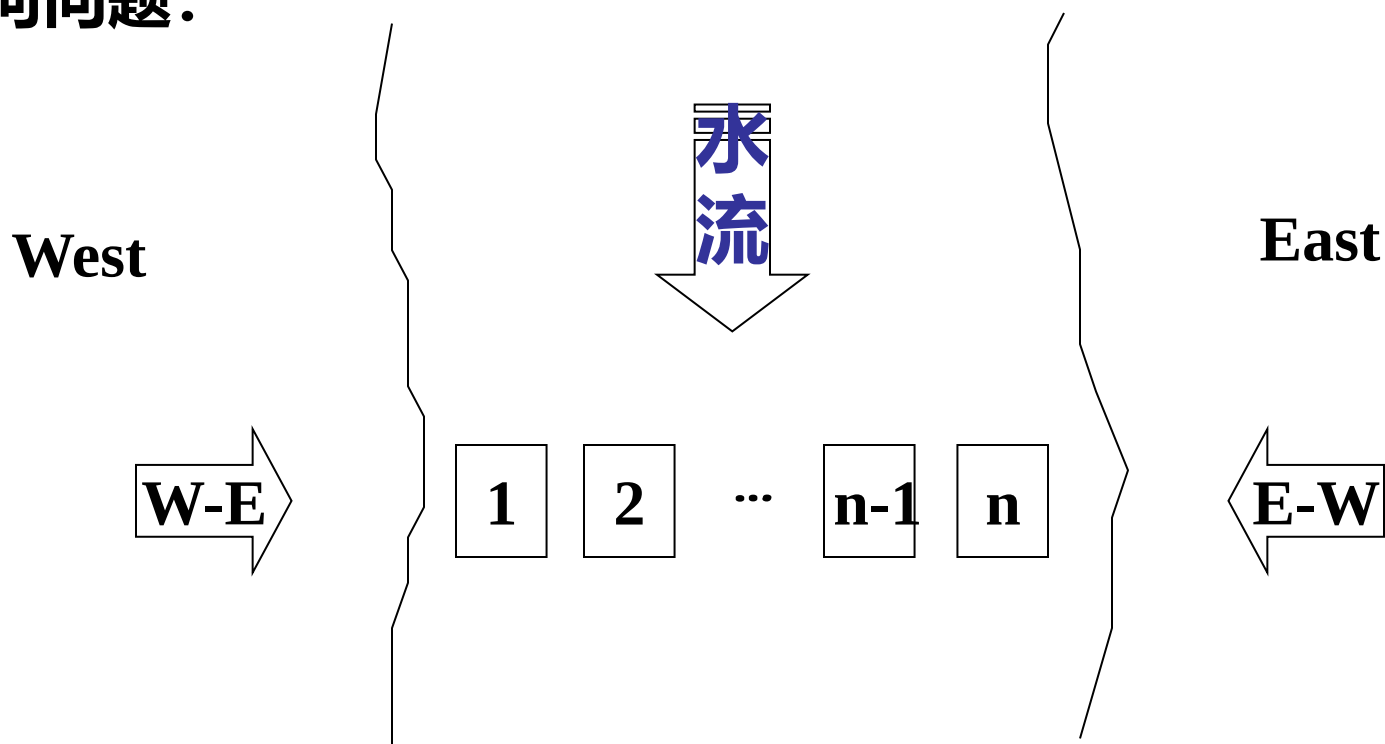
```
begin
  for I:=0 to 4 do
    begin state[I]:=thinking; self[I].value:=0 end;
  mutex.value:=1;
  cobegin
    ph0: philosopher(0);
    .....
    Ph4: philosopher(4);
  coend
end.
```

哲学家就餐问题讨论

- 何时发生死锁？
- 怎样从死锁中恢复？
- 怎样避免死锁的发生？
- 如何预防死锁？

死锁的例子5-10：过河问题

过河问题：



Deadlock prevention: one direction at any time.

死锁的例子5-10：过河问题

```
Var west_crossing,east_crossing:integer; (0,0)
    west_wait, east_wait:integer; (0,0);
    wq, eq: semaphore;
    mutex:semaphore;
```

西面过河者活动：

```
P(mutex);
```

```
If east_crossing>0 Then
```

```
Begin
```

```
    west_wait:=west_wait+1;
```

```
    V(mutex);
```

```
    P(wq)
```

```
End;
```

```
Else Begin
```

```
    west_crossing:=west_crossing+1;
```

死锁的例子5-10：过河问题

V(mutex)

End;

过河;

P(mutex);

west_crossing:=west_crossing-1;

If west_crossing=0 Then

While east_wait >0 Do

Begin

east_wait:=east_wait-1;

east_crossing:=east_crossing+1;

V(eq);

End;

V(mutex);

死锁的例子5-10：过河问题

东面过河者活动：

P(mutex);

If west_crossing>0 Then

Begin

east_wait:=east_wait+1;

V(mutex);

P(eq)

End

Else Begin

east_crossing:=east_crossing+1;

V(mutex)

End;

死锁的例子5-10：过河问题

过河；

P(mutex);

east_crossing:=east_crossing-1;

If east_crossing=0 Then

While west_wait >0 Do

Begin

west_wait:=west_wait-1;

west_crossing:=west_crossing+1;

V(wq);

End;

V(mutex);

思考问题

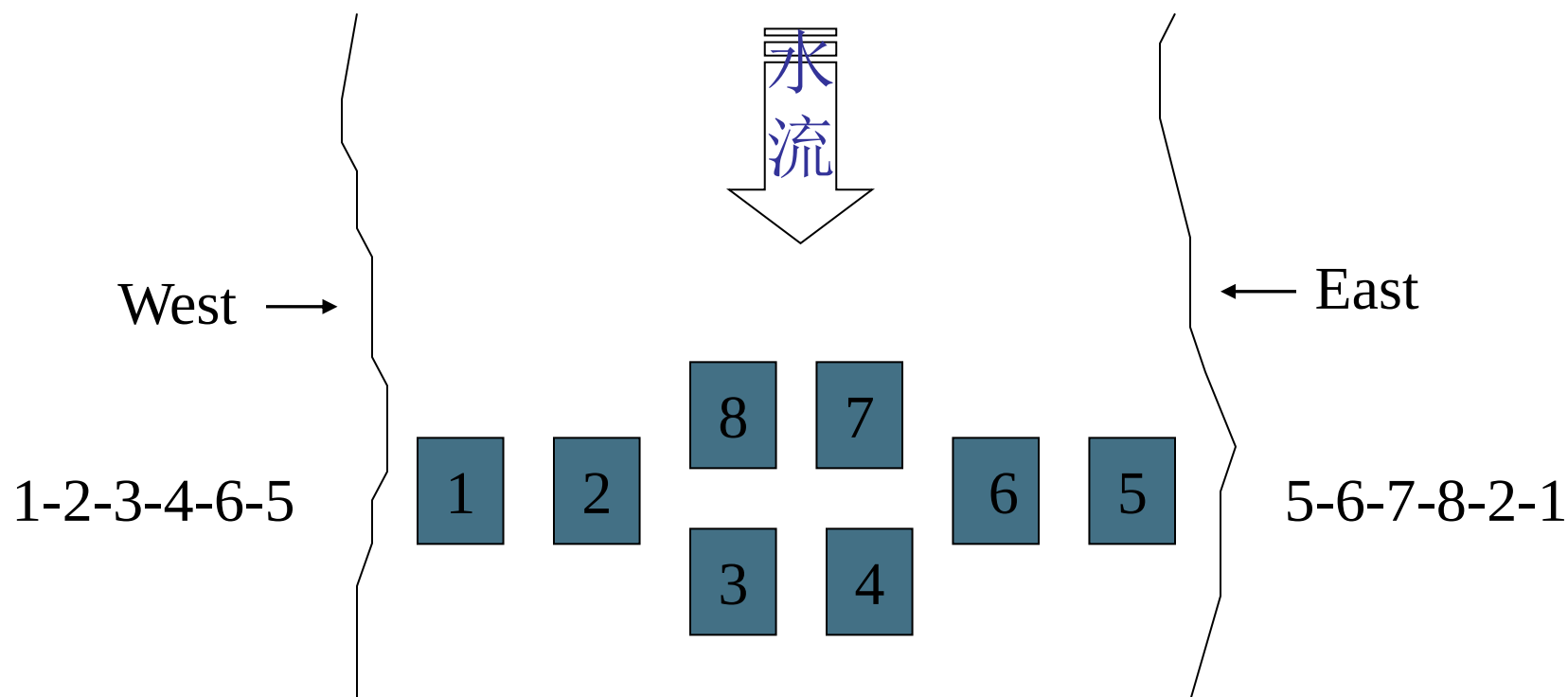
- 对于过河问题，考虑一个没有饿死情况的解法。

例. 过河问题

■ 问题描述

- 有南北走向的河流如下图所示，河中用石块搭成的便桥
- 每个石块上最多容纳一个过河者，两个相邻石块的间距恰好为一步
- 西岸过河者经过石块1、2、3、4、6、5到达东岸，东岸过河者经过石块5、6、7、8、2、1到达东岸
- 试分析可能发生的死锁情况，给出一个无死锁、无饥饿、并行度高的解法，并用信号量和PV操作实现

例. 过河问题



要求： (1) 无死锁； (2) 无饿死； (3) 并行度高。

问题分析

■ 死锁分析

- 当2个方向各有3人踏上石块时，必发生死锁
- 当2个方向各有1人分别踏上1、2或者5、6时，也会发生死锁

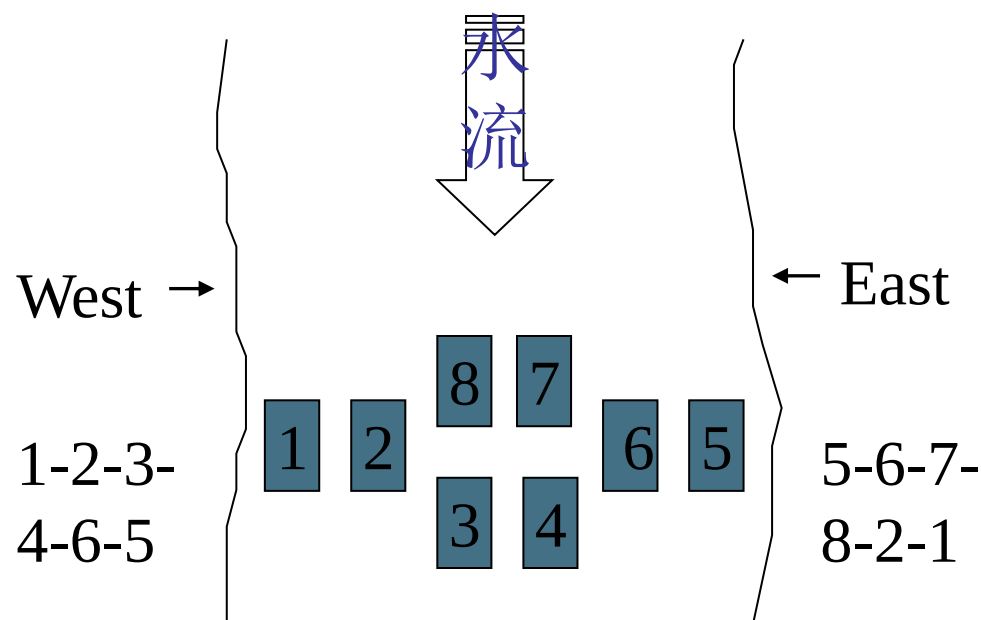
■ 死锁预防

- 最简单的方式规定东、西两岸人员不能同时过河，但这可能导致饥饿或饿死，同时也影响并行度
- 根据资源数量，限制同时过河的人数在5个以内，这时至少有一个方向的过河人数不超过2个，当他们分别踏上3、4或者7、8石块时，对另一方向的过河人无影响
- 其次，对两岸竞争的1、2和5、6两对石块，采用有序分配法，即按1、2和5、6次序申请

```
Var S, s1, s2, s3, s4, s5, s6, s7, s8: semaphore;  
(5, 1, 1, 1, 1, 1, 1, 1, 1)
```

W→E:

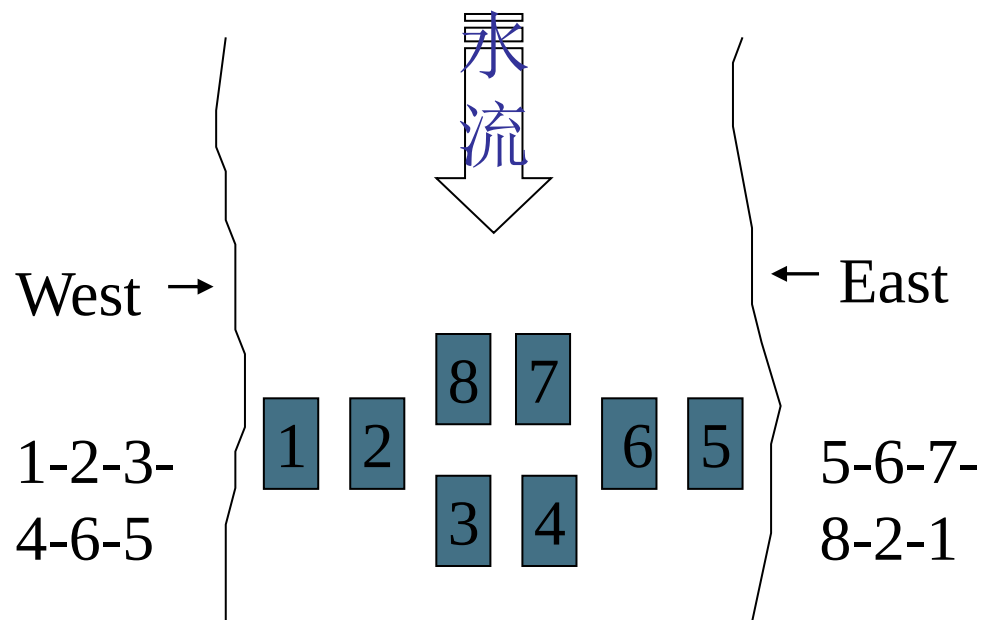
```
P(S);      V(s3);  
P(s1);      P(s5);  
走到1;      P(s6);  
P(s2);      走到6;  
走到2;      V(s4);  
V(s1);      走到5;  
P(s3);      V(s6);  
走到3;      走到E;  
V(s2);      V(s5);  
P(s4);      V(S);  
走到4;
```



```
Var S, s1, s2, s3, s4, s5, s6, s7, s8: semaphore;  
(5, 1, 1, 1, 1, 1, 1, 1, 1)
```

E→W:

```
P(S);      V(s7);  
P(s5);      P(s1);  
走到5;      P(s2);  
P(s6);      走到2;  
走到6;      V(s8);  
V(s5);      走到1;  
P(s7);      V(s2);  
走到7;      走到W;  
V(s6);      V(s1);  
P(s8);      V(S);  
走到8;
```



问题分析

- 对于过河问题上述解法正确但并不能保证最大并行度，实际上当一个方向过河人数不超过2人时，另一方向过河人数可以达到6而不发生死锁，此时系统最大并行度可以达到8，解法如下。
- semaphore S; //初值为8
 semaphore mutex; //初值为1
 semaphore wq, eq; //初值为0
 int ec, wc; // 初值为0
 int ew, ww; // 初值为0

semaphore s1,s2,s3,s4,s5,s6,s7,s8;// (1,1,1,1,1,1,1,1)

W→E:	P(s1);	V(s5);	P(mutex);
P (S) ;	走到1;	P(s3);	wc--;
P(mutex);	P(s2);	P(s4);	if wc==2;
if ec>=3	走到2;	走到4;	while ew>0
&&wc==2;	V(s1);	V(s6);	ew--;
ww++;	P(s5);	走到3;	ec++;
V(mutex);	走到5;	V(s4);	V(eq);
P(wq);	V(s2);	走到E;	end;
else	P(s6);	V(s3);	else if wc==1 && ww>0
wc++;	走到6	V (S) ;	ww--;wc++;
V(mutex);			V(wq); end
end			end V(mutex)

E→W:

```

P (S) ;
P(mutex);
if wc>=3
&&ec==2;
    ew++;
    V(mutex);
    P(eq);
else
    ec++;
    V(mutex);
end
    
```

```

P(s3);
走到3;
P(s4);
走到4;
V(s3);
P(s7);
走到7;
V(s4);
P(s8);
走到8
    
```

```

V(s7);
P(s1);
P(s2);
走到2;
V(s8);
走到1;
V(s2);
走到W;
V(s1);
V (S) ;
    
```

```

P(mutex);
ec--;
if ec==2;
    while ww>0
        ww--;
        wc++;
        V(wq);
    end;
else if ec==1 && ew>0
    ew--;ec++;
    V(eq); end
end
V(mutex)
    
```

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.13 简单组合资源死锁的静态分析

- 可复用资源(reusable resource)
 - 一次只能分配给一个进程使用的资源
- 组合资源
 - 由相对独立的若干资源构成的资源集合，其中每个相对独立的资源称为子资源
- 同种组合资源
 - 由相同类型的子资源构成的组合资源.
- 每类子资源只有一个资源实例称为简单组合资源。

5.13 简单组合资源死锁的静态分析

条件：已知各个进程有关资源的活动序列

判断：有无死锁可能性

- 步骤1：以每个进程占有资源，申请资源作为一个状态，
记作：(p_i : a_j : a_{k1}, ..., a_{k_n}) = (进程：请求：占有)；
- 步骤2：以每个状态为一个节点；
- 步骤3：如p₁所申请资源为p₂所占有，则由p₁向p₂画一有向弧(相同进程间不画)；
- 步骤4：找出所有环路；
- 步骤5：判断环路上状态是否能同时到达，如有死锁可能性，否则无死锁可能性。
 - (1) 环路中有相同进程，不能到达；
 - (2) 环路中有相同被占有资源，不能到达。

死锁分析例子

$R = \{A, B, C, D, E, F, G\}$

p1: c d $\bar{c}$ a b $\bar{d}$ $\bar{a}$ $\bar{b}$

p2: d e $\bar{d}$ b f $\bar{e}$ $\bar{b}$ $\bar{f}$

p3: c e $\bar{e}$ f a $\bar{e}$ $\bar{f}$ $\bar{a}$

(p1:d:c)

(p2:e:d)

(p3:e:c)

(p1:a:d)

(p2:b:e)

(p3:f:c)

(p1:b:da)

(p2:f:eb)

(p3:a:cf)

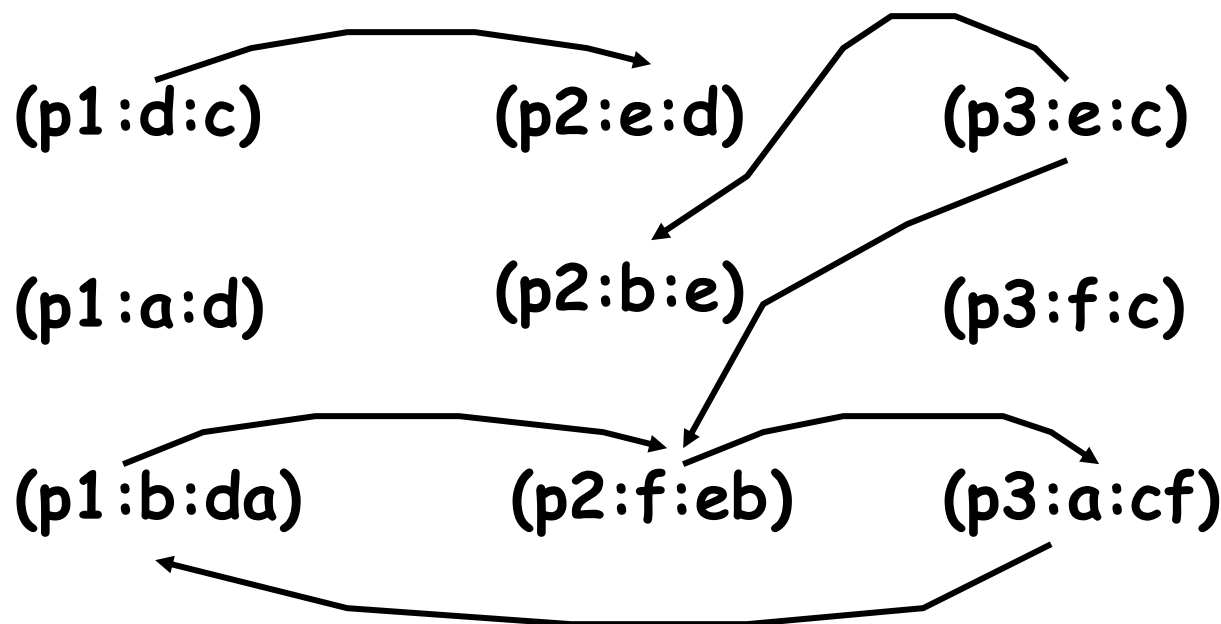
死锁分析例子 (Cont.)

$R = \{A, B, C, D, E, F, G\}$

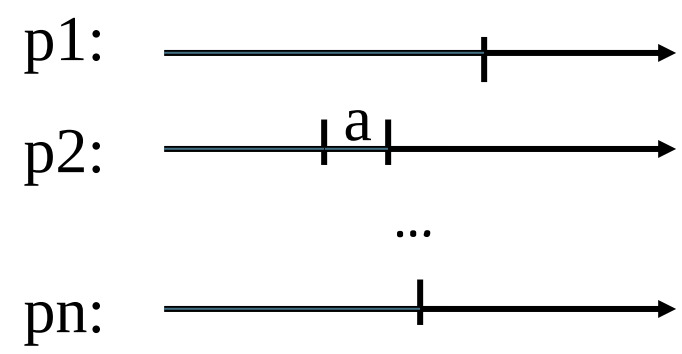
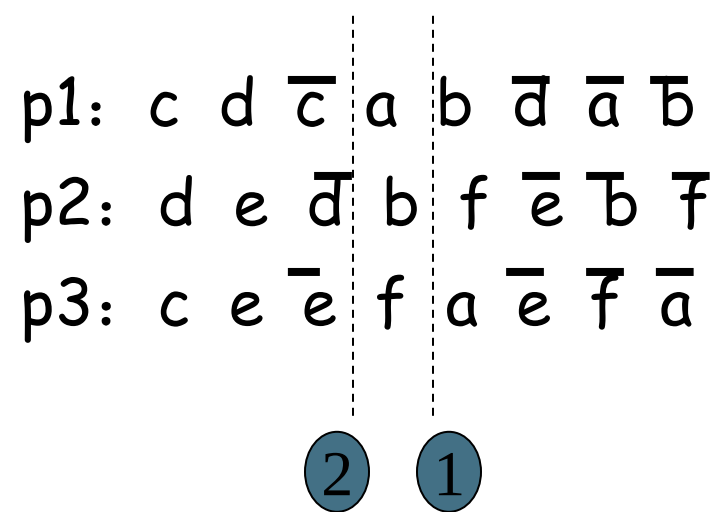
p1: c d $\bar{c}$ a b $\bar{d}$ $\bar{a}$ $\bar{b}$

p2: d e $\bar{d}$ b f $\bar{e}$ $\bar{b}$ $\bar{f}$

p3: c e $\bar{e}$ f a $\bar{e}$ $\bar{f}$ a



死锁分析例子 (Cont.)



证明：线1不可达；

反证：假定线1可达，则线2可达。

p2先于p1; p3先于p2; p1先于p3, 矛盾。

例：死锁静态分析

设有 7 个简单资源：A、B、C、D、E、F、G，各资源申请命令分别为 a、b、c、d、e、f、g，释放命令分别为 $\bar{a}$ 、 $\bar{b}$ 、 $\bar{c}$ 、 $\bar{d}$ 、 $\bar{e}$ 、 $\bar{f}$ 、 $\bar{g}$ 。设系统有 P1、P2、P3 三个进程，各进程有关资源的活动序列为：

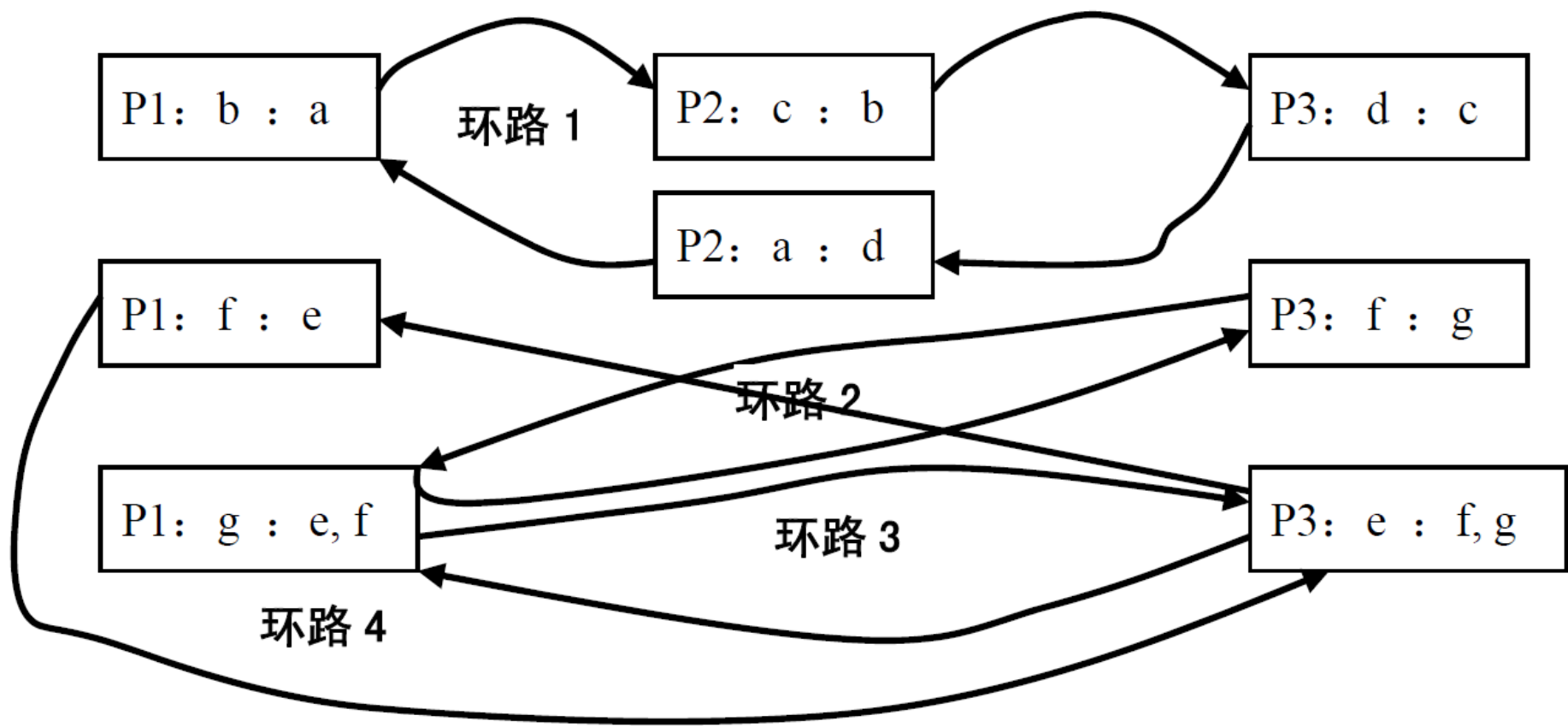
P1 活动：a b $\bar{a}$ $\bar{b}$ e f g $\bar{e}$ $\bar{f}$ $\bar{g}$

P2 活动：b c $\bar{b}$ $\bar{c}$ d a $\bar{d}$ $\bar{a}$

P3 活动：c d $\bar{c}$ $\bar{d}$ g f e $\bar{e}$ $\bar{f}$ $\bar{g}$

分析当 P1、P2、P3 并发执行时，是否有发生死锁的可能性，并说明原因。

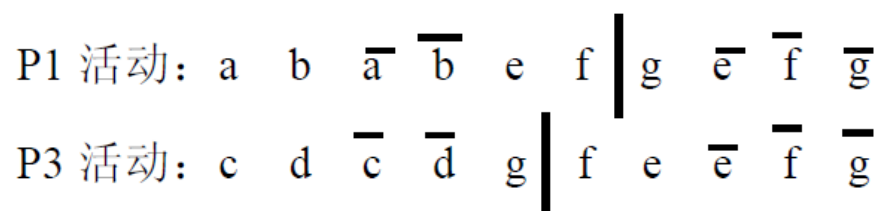
解：(1) (4 分) 根据三个进程有关资源的活动，画出进程资源申请、占用状态图如下：



(2) (6分) 环路分析:

环路 1: 该环路中有相同的进程 P2, 三个进程并发执行时, 不会发生此种情况的死锁;

环路 2: 该环路反映的 P1、P3 的运行情况是两个进程同时执行到竖线位置 (如下图)

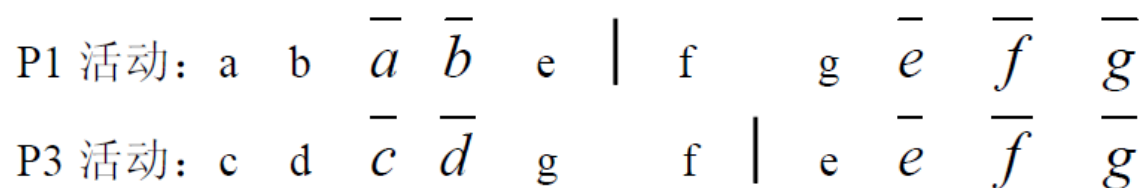


两个进程能同时执行到竖线位置, 此时

P1 占有资源 E 和 F, 要申请资源 G; P3 占有资源 G, 要申请资源 F。出现死锁;

环路 3: 该环路反映的运行状态是进程 P1 和 P3 同时占有资源 F, 但由于资源 F 只有一个, 因此此种运行状态不可能出现, 即不会发生此种情况的死锁。

环路 4: 该环路反映的 P1、P3 的运行情况是两个进程同时执行到竖线位置 (如下图)



两个进程能同时执行到竖线位置, 此时

P1 占有资源 E, 要申请资源 F; P3 占有资源 G、F, 要申请资源 E。出现死锁

提纲

5.1 死锁的概念

5.2 死锁类型

5.3 死锁的条件

5.4 资源分配图

5.5 死锁的处理

5.6 死锁预防

5.7 死锁避免

5.8 死锁的发现

5.9 死锁的恢复

5.10 鸵鸟算法

5.11 有关问题的讨论

5.12 饥饿与活锁

5.13 简单组合资源死锁
的静态分析

5.14 同种组合资源死锁
的必要条件

5.14 同种组合资源死锁的必要条件

M: 资源数量

N: 使用该类资源进程的数量

Σ : 所有进程所需要该类资源的总量

假定死锁, n 个进程参与了死锁 ($2 \leq n \leq N$)

参与死锁的进程所需资源的总量 $\geq M+n$

未参与死锁进程所需资源的总量 $\geq N-n$

所有进程所需资源的总量 $\Sigma \geq M+n+N-n=M+N$

当 $\Sigma < M+N$ 时, 一定没有死锁;

当 $\Sigma \geq M+N$ 时, 至少有一个交叉有死锁。

例. 同种组合资源死锁

例1. $M=15$;

$N=4$; $P1(4)$; $P2(6)$; $P3(1)$; $P4(7)$

$$\Sigma=4+6+1+7=18 < M+N=19$$

无死锁可能性。

例2. $M=15$;

$N=4$; $P1(5)$; $P2(6)$; $P3(1)$; $P4(7)$

$$\Sigma=5+6+1+7=19 \geq M+N=19$$

有死锁可能性。死锁情况：

P1: a a a a | a
 P2: a a a a a | a
 P3: { a
 P4: a a a a a a | a

作业 #3

1. 第5章课后习题9
2. 第5章课后习题10
3. 设系统有R1和R2两类独占型资源供进程P1和P2所共享，R1有2台，R2有1台。已知P1和P2均以如下的顺序申请和释放资源：

申请R1； 申请R2； 申请R1； 释放R1； 释放R2； 释放R1。

问 题： 根据进程P1和P2对资源的请求序列，
画图分析两进程并发运行时可能到达的所有死锁点。